

JAVA 17 OCP · YDS STUDY SERIES

Unit 03 · Making Decisions · Eksiksiz Çift Dilli Ana Not

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu belge, yüklenen OCP Java SE 17 kaynağındaki bölüm metnini kaynak sırasını koruyarak işler. Her düzeltilmiş English parça hemen ardından doğal Türkçe karşılığıyla verilir; kod yalnız bir kez gösterilir. Ayrıntılı dil çalışması için [vocabulary](#) ve [grammar notes](#) kaynaklarına bakın.

Kapsam manifesti

- Kaynak: `exam_lecture/OCP_Java_SE17_Chapter1den_Itibaren.pdf`
- Bölüm: Chapter 3 · Making Decisions
- PDF sayfaları: 0101–0154 (dahil, 54 sayfa)
- Korunan temiz kaynak satırı: 1837
- Çıkarılan öğeler: yalnız tekrarlanan running header/footer ve PDF sayfa numarası; soru numarasıyla birleşmiş üstbilgilerde soru numarası korunmuştur.
- OCR düzenlemeleri: soft-hyphen, bölünmüş sözcük, `->`, `--`, `==` ve tarih ayırıcıları teknik yazıma getirilmiştir.
- İzlenebilirlik: Her kaynak sayfası `source-page` marker'ı, korunan satır sayısı ve kısa SHA-256 özeti taşır.

İçindekiler

- 1 [Creating Decision-Making Statements](#)
- 2 [Applying switch Statements](#)
- 3 [Writing while Loops](#)
- 4 [Constructing for Loops](#)
- 5 [Controlling Flow with Branching](#)
- 6 [Summary](#)
- 7 [Exam Essentials](#)
- 8 [Review Questions](#)
- 9 [Kaynak dışı çözüm ve teknik pekiştirme appendix'i](#)
- 10 [Page coverage ledger](#)

Kaynak sırasındaki çift dilli içerik

English: Chapter 3 Türkçe: Bölüm 3

Making Decisions

Türkçe başlık: Karar Verme

OCP EXAM OBJECTIVES COVERED IN

Türkçe başlık: BU BÖLÜMDE KAPSANAN OCP SINAV HEDEFLERİ

English: THIS CHAPTER: **Türkçe:** BU BÖLÜM:

[x] [x] Controlling Program Flow

Türkçe başlık: [x] [x] Program Akışını Kontrol Etme

English: ■ ■ Create program flow control constructs including if/else, switch statements and expressions, loops, and break and continue statements **Türkçe:** ■ ■ `if/else`, `switch` statement ve expression'ları, loop'lar ile `break` ve `continue` statement'ları dahil program akışı kontrol yapıları oluşturun.

[x] [x] Utilizing Java Object-Oriented Approach

Türkçe başlık: [x] [x] Java Nesne Yönelimli Yaklaşımını Kullanma

English: ■ ■ Implement polymorphism and differentiate object type versus reference type. Perform type casting, identify object types using instanceof operator and pattern matching **Türkçe:** ■ ■ Polymorphism uygulayın; object type ile reference type arasındaki farkı belirleyin. Type casting yapın; `instanceof` operator'ü ve pattern matching kullanarak object type'lerini tanımlayın.

English: Like many programming languages, Java is composed primarily of variables, operators, and statements put together in some logical order. In the last chapter, we covered how to create and manipulate variables. Writing software is about more than managing variables, though; it is about creating applications that can make intelligent decisions. In this chapter, we present the various decision-making statements available to you within the language. This knowledge will allow you to build complex functions and class structures that you'll see throughout this book. **Türkçe:** Birçok programlama dili gibi Java da temel olarak mantıksal bir sırayla bir araya getirilen variable, operator ve statement'lardan oluşur. Önceki bölümde variable'ların nasıl oluşturulacağını ve değiştirileceğini ele aldık. Ancak yazılım geliştirmek yalnızca variable yönetmekten ibaret değildir; akıllı kararlar verebilen uygulamalar oluşturmayı da gerektirir. Bu bölümde Java dilindeki çeşitli decision-making statement'larını tanıtıyoruz. Bu bilgiler, kitap boyunca göreceğiniz karmaşık function ve class yapılarını kurmanızı sağlayacaktır.

Creating Decision-Making Statements

Türkçe başlık: Decision-Making Statement'ları Oluşturma

English: Java operators allow you to create a lot of complex expressions, but they're limited in the manner in which they can control program flow. Imagine you want a method to be executed only under certain conditions that cannot be evaluated until runtime. For example, on rainy days, a zoo should remind patrons to bring an umbrella, or on a snowy day, the zoo might need to close. The software doesn't change, but the behavior of the software should, depending on the inputs supplied in the moment. In this section, we discuss decision-making statements including if and else, along with the new pattern matching feature. **Türkçe:** Java operatör'leri çok sayıda karmaşık expression oluşturmanıza izin verir; ancak program akışını kontrol etme biçimleri sınırlıdır. Yalnızca runtime'da değerlendirilebilen belirli koşullar altında bir method'un çalıştırılmasını istediğinizi düşünün. Örneğin, yağmurlu günlerde bir hayvanat bahçesi ziyaretçilere şemsiye getirmelerini hatırlatabilir; karlı bir günde ise kapanması gerekebilir. Yazılım değişmez, fakat o anda sağlanan input'lara bağlı olarak davranışı değişmelidir. Bu bölümde `if`, `else` ve yeni pattern matching özelliği dahil decision-making statement'larını ele alıyoruz.

Statements and Blocks

Türkçe başlık: Statement'lar ve Block'lar

English: As you may recall from Chapter 1, "Building Blocks," a Java statement is a complete unit of execution in Java, terminated with a semicolon (;). In this chapter, we introduce you to various Java control flow statements. Control flow statements break up the flow of execution by using decision-making, looping, and branching, allowing the application to selectively execute particular segments of code.

Türkçe: Bölüm 1, "Building Blocks"tan hatırlayabileceğiniz gibi Java statement, noktalı virgülle (;) sonlanan eksiksiz bir yürütme birimidir. Bu bölümde çeşitli Java control-flow statement'larını tanıtıyoruz. Control-flow statement'ları decision-making, looping ve branching kullanarak yürütme akışını böler; böylece uygulama belirli kod bölümlerini seçerek çalıştırabilir.

English: These statements can be applied to single expressions as well as a block of Java code. **Türkçe:** Bu statement'lar tek bir expression'a veya bir Java kod block'una uygulanabilir.

English: As described in Chapter 1, a block of code in Java is a group of zero or more statements between balanced braces ({}) and can be used anywhere a single statement is allowed. For example, the following two snippets are equivalent, with the first being a single expression and the second being a block containing the same statement: **Türkçe:** Bölüm 1'de açıklandığı gibi Java'da kod block'u, dengeli küme parantezleri ({}) arasındaki sıfır veya daha fazla statement'tan oluşur ve tek bir statement'a izin verilen her yerde kullanılabilir. Örneğin aşağıdaki iki kod parçası eşdeğerdir: ilki tek bir statement, ikincisi ise aynı statement'ı içeren bir block'tur.

```
// Single statement
patrons++;
```

```
// Statement inside a block
{
    patrons++;
}
```

English: A statement or block often serves as the target of a decision-making statement. For example, we can prepend the decision-making if statement to these two examples: **Türkçe:** Bir statement veya block çoğu zaman bir decision-making statement'ın hedefidir. Örneğin bu iki örneğin başına decision-making if statement'ını ekleyebiliriz:

```
// Single statement
if(ticketsTaken > 1)
    patrons++;
// Statement inside a block
if(ticketsTaken > 1)
{
    patrons++;
}
```

English: Again, both of these code snippets are equivalent. Just remember that the target of a decision-making statement can be a single statement or block of statements. For the rest of the chapter, we use both forms to better prepare you for what you will see on the exam. **Türkçe:** Bu iki kod parçası yine eşdeğerdir. Bir decision-making statement'ın hedefinin tek bir statement veya bir statement block'u olabileceğini unutmayın. Sınavda karşılaşacağınız biçimlere hazırlanmanız için bölümün devamında her ikisini de kullanacağız.

English: While both of the previous examples are equivalent, stylistically using blocks is often preferred, even if the block has only one statement. The second form has the advantage that you can quickly insert new lines of code into the block, without modifying the surrounding structure. **Türkçe:** Önceki iki örnek eşdeğer olsa da block yalnızca bir statement içerdiğinde bile stil açısından block kullanımı çoğunlukla tercih edilir. İkinci biçim, çevresindeki yapıyı değiştirmeden block'a hızla yeni kod satırları ekleyebilme avantajı sağlar.

The if Statement

Türkçe başlık: if Statement

English: Often, we want to execute a block only under certain circumstances. The if statement, as shown in Figure 3.1, accomplishes this by allowing our application to execute a particular block of code if and only if a boolean expression evaluates to true at runtime. **Türkçe:** Çoğu zaman bir block'u yalnızca belirli koşullarda çalıştırmak isteriz. Şekil 3.1'de gösterilen if statement, bir boolean expression runtime'da true değerini ancak ve ancak ürettiğinde uygulamanın ilgili kod block'unu çalıştırmasını sağlar.

FIGURE 3.1 The structure of an if statement

Türkçe başlık: ŞEKİL 3.1 · Bir if statement'ın yapısı

English: if keyword Parentheses (required) **Türkçe:** if keyword'ü · Parantez zorunludur

```
if (booleanExpression) {
```

English: Curly braces required for block of multiple statements, optional for single statement **Türkçe:** Birden çok statement içeren block için küme parantezleri zorunlu, tek statement için isteğe bağlıdır.

```
}
```

English: For example, imagine we had a function that used the hour of day, an integer value from 0 to 23, to display a message to the user: **Türkçe:** Örneğin kullanıcıya mesaj göstermek için günün saatini, yani 0 ile 23 arasındaki bir integer değeri kullanan bir function düşünün:

```
if(hourOfDay < 11)
    System.out.println("Good Morning");
```

English: If the hour of the day is less than 11, then the message will be displayed. Now let's say we also wanted to increment some value, morningGreetingCount, every time the greeting is printed. We could write the if statement twice, but luckily Java offers us a more natural approach using a block: **Türkçe:** Günün saati 11'den küçükse mesaj gösterilir. Şimdi de selamlama her yazdırıldığında morningGreetingCount değerini artırmak istediğimizi varsayalım. if statement'ını iki kez yazabiliriz; ancak Java block kullanarak daha doğal bir yaklaşım sunar:

```
if(hourOfDay < 11) {
    System.out.println("Good Morning");
    morningGreetingCount++;
}
```

Watch Indentation and Braces

Türkçe başlık: Girinti ve Küme Parantezlerine Dikkat

English: One area where the exam writers will try to trip you up is if statements without braces {}. For example, take a look at this slightly modified form of our example: **Türkçe:** Sınav yazarlarının sizi yanıltmaya çalışabileceği konulardan biri, küme parantezi ({}) kullanılmayan if statement'larıdır. Örneğimizin biraz değiştirilmiş şu biçimine bakın:

```
if(hourOfDay < 11)
    System.out.println("Good Morning");
    morningGreetingCount++;
```

English: Based on the indentation, you might be inclined to think the variable morningGreetingCount is only going to be incremented if hourOfDay is less than 11, but that's not what this code does. It will execute the print statement only if the condition is met, but it will always execute the increment operation. **Türkçe:** Girintiyi bakarak morningGreetingCount variable'ının yalnızca hourOfDay 11'den küçük olduğunda artırılacağını düşünebilirsiniz; fakat kod böyle çalışmaz. Print statement yalnızca koşul sağlanırsa çalışır, increment işlemi ise her durumda çalışır.

English: Remember that in Java, unlike some other programming languages, tabs are just whitespace and are not evaluated as part of the execution. When you see a control flow statement in a question, be sure to trace the open and close braces of the block, ignoring any indentation you may come across.

Türkçe: Bazı programlama dillerinin aksine Java'da tab karakterleri yalnızca whitespace'tir ve yürütmenin bir parçası olarak değerlendirilmez. Bir soruda control-flow statement gördüğünüzde girintiyi dikkate almadan block'un açılış ve kapanış küme parantezlerini izleyin.

The else Statement

Türkçe başlık: `else` Statement

English: Let's expand our example a little. What if we want to display a different message if it is 11 a.m. or later? Can we do it using only the tools we have? Of course we can! **Türkçe:** Örneğimizi biraz genişletelim. Saat 11.00 veya daha ileriye farklı bir mesaj göstermek istersek ne olur? Elimizdeki araçlarla bunu yapabilir miyiz? Elbette!

```
if(hourOfDay < 11) {  
    System.out.println("Good Morning");  
}
```

```
}  
if(hourOfDay >= 11) {  
    System.out.println("Good Afternoon");  
}
```

English: This seems a bit redundant, though, since we're performing an evaluation on `hourOfDay` twice. Luckily, Java offers us a more useful approach in the form of an `else` statement, as shown in Figure 3.2.

Türkçe: `hourOfDay` iki kez değerlendirildiği için bu yaklaşım biraz gereksiz görünür. Neyse ki Java, Şekil 3.2'de gösterildiği gibi `else` statement biçiminde daha kullanışlı bir yaklaşım sunar.

FIGURE 3.2 The structure of an else statement

Türkçe başlık: ŞEKİL 3.2 · Bir `else` statement'ın yapısı

English: `if` keyword Parentheses (required) **Türkçe:** `if` keyword'ü · Parantez zorunludur

```
if (booleanExpression) {  
    // Branch if true  
} else {
```

English: Curly braces required for block of multiple statements, optional for single statement **Türkçe:** Birden çok statement içeren block için küme parantezleri zorunlu, tek statement için isteğe bağlıdır.

```
// Branch if false
```

English: Optional else statement **Türkçe:** İsteğe bağlı `else` statement

```
}
```

English: Let's return to this example: **Türkçe:** Örneğimize dönelim:

```
if(hourOfDay < 11) {  
    System.out.println("Good Morning");  
} else System.out.println("Good Afternoon");
```

English: Now our code is truly branching between one of the two possible options, with the boolean evaluation happening only once. The else operator takes a statement or block of statements, in the same manner as the if statement. Similarly, we can append additional if statements to an else block to arrive at a more refined example: **Türkçe:** Artık kod iki olası seçenekten birine gerçekten dallanır ve boolean değerlendirme yalnızca bir kez yapılır. **else**, **if** statement gibi tek bir statement veya statement block'u alır. Benzer biçimde daha ayrıntılı bir akış oluşturmak için bir **else** block'una ek **if** statement'ları bağlayabiliriz:

```
if(hourOfDay < 11) {  
    System.out.println("Good Morning");  
} else if(hourOfDay < 15) {  
    System.out.println("Good Afternoon");  
} else {  
    System.out.println("Good Evening");  
}
```

English: In this example, the Java process will continue execution until it encounters an if statement that evaluates to true. If neither of the first two expressions is true, it will execute the final code of the else block. **Türkçe:** Bu örnekte Java, sonucu **true** olan bir **if** statement'ıyla karşılaşana kadar yürütmeyi sürdürür. İlk iki expression da **true** değilse son **else** block'undaki kodu çalıştırır.

Verifying That the if Statement Evaluates to a Boolean Expression

Türkçe başlık: **if** Statement'ının Boolean Expression Ürettiğini Doğrulama

English: Another common way the exam may try to lead you astray is by providing code where the boolean expression inside the if statement is not actually a boolean expression. For example, take a look at the following lines of code: **Türkçe:** Sınavın sizi yanıltabileceği yaygın yöntemlerden biri, **if** statement'ındaki expression'ın gerçekte boolean olmamasıdır. Örneğin aşağıdaki kod satırlarına bakın:

```
int hourOfDay = 1;  
if(hourOfDay) { // DOES NOT COMPILE
```

English: ... **Türkçe:** ...

```
}
```

English: This statement may be valid in some other programming and scripting languages, but not in Java, where 0 and 1 are not considered boolean values. **Türkçe:** Bu statement başka bazı programlama ve scripting dillerinde geçerli olabilir; ancak 0 ile 1 Java'da boolean değer sayılmadığı için Java'da geçerli değildir.

Shortening Code with Pattern Matching

Türkçe başlık: pattern matching ile Kodu Kısaltma

English: Java 16 officially introduced pattern matching with if statements and the instanceof operator.

Türkçe: Java 16, if statement'ları ve instanceof operator'üyle pattern matching'i resmen kullanıma sundu.

English: Pattern matching is a technique of controlling program flow that only executes a section of code that meets certain criteria. It is used in conjunction with if statements for greater program control. **Türkçe:** Pattern matching, yalnızca belirli ölçütleri karşılayan bir kod bölümünü çalıştırarak program akışını kontrol etme tekniğidir. Program üzerinde daha ayrıntılı kontrol sağlamak için if statement'larıyla birlikte kullanılır.

English: If pattern matching is new to you, be careful not to confuse it with the Java Pattern class or regular expressions (regex). While pattern matching can include the use of regular expressions for filtering, they are unrelated concepts. **Türkçe:** Pattern matching sizin için yeniyse bunu Java Pattern class'ı veya regular expression'larla (regex) karıştırmamaya dikkat edin. Pattern matching kapsamında filtreleme amacıyla regular expression kullanılabilir de bunlar birbirinden farklı kavramlardır.

English: Pattern matching is a new tool at your disposal to reduce boilerplate in your code. **Türkçe:** Pattern matching, kodunuzdaki boilerplate code miktarını azaltmak için kullanabileceğiniz yeni bir araçtır.

English: Boilerplate code is code that tends to be duplicated throughout a section of code over and over again in a similar manner. A lot of the newer enhancements to the Java language focus on reducing boilerplate code. **Türkçe:** Boilerplate code, bir kod bölümü boyunca benzer biçimde tekrar tekrar kopyalanan koddur. Java dilindeki yeni geliştirmelerin çoğu boilerplate code'u azaltmaya odaklanır.

English: To understand why this tool was added, consider the following code that takes a Number instance and compares it with the value 5. If you haven't seen Number or Integer, you just need to know that Integer inherits from Number for now. You'll see them a lot in this book! **Türkçe:** Bu aracın neden eklendiğini anlamak için bir Number instance'ı alıp onu 5 değeriyle karşılaştıran aşağıdaki kodu düşünün. Number veya Integer ile henüz karşılaşmadıysanız şimdilik Integer'ın Number'dan miras aldığını bilmeniz yeterlidir. Kitap boyunca bu type'ları sıkça göreceksiniz.

```
void compareIntegers(Number number) {  
    if(number instanceof Integer) {  
        Integer data = (Integer)number;  
    }  
}
```

```
System.out.print(data.compareTo(5));
}
}
```

English: The cast is needed since the `compareTo()` method is defined on `Integer`, but not on `Number`.

Türkçe: `compareTo()` method'u `Integer` üzerinde tanımlı, `Number` üzerinde tanımlı olmadığı için cast gerekir.

English: Code that first checks if a variable is of a particular type and then immediately casts it to that type is extremely common in the Java world. It's so common that the authors of Java decided to implement a shorter syntax for it: **Türkçe:** Öncelikle bir değişkenin belirli bir türde olup olmadığını kontrol eden ve ardından onu hemen bu türe aktaran kod, Java dünyasında oldukça yaygındır. Bu o kadar yaygın ki, Java yazarları bunun için daha kısa bir sözdizimi uygulamaya karar verdiler:

```
void compareIntegers(Number number) {
    if(number instanceof Integer data) {
        System.out.print(data.compareTo(5));
    }
}
```

English: The variable `data` in this example is referred to as the pattern variable. Notice that this code also avoids any potential `ClassCastException` because the cast operation is executed only if the implicit `instanceof` operator returns `true`. **Türkçe:** Bu örnekteki `data` variable'ına pattern variable denir. Cast işlemi yalnızca örtük `instanceof` operator'ü `true` döndürürse yapıldığı için kodun olası bir `ClassCastException`ı da önlediğine dikkat edin.

Reassigning Pattern Variables

Türkçe başlık: Pattern Variable'lara Yeniden Değer Atama

English: While possible, it is a bad practice to reassign a pattern variable since doing so can lead to ambiguity about what is and is not in scope. **Türkçe:** Mümkün olsa da pattern variable'a yeniden değer atamak kötü bir uygulamadır; çünkü bu, neyin scope içinde olduğu konusunda belirsizliğe yol açabilir.

```
if(number instanceof Integer data) {
    data = 10;
}
```

English: The reassignment can be prevented with a `final` modifier, but it is better not to reassign the variable at all. **Türkçe:** Yeniden atama, `final` değiştiriciyle önlenabilir, ancak değişkeni hiç yeniden atamamak daha iyidir.

```
if(number instanceof final Integer data) {
    data = 10; // DOES NOT COMPILE
}
```

Pattern Variables and Expressions

Türkçe başlık: Pattern Variable'lar ve Expression'lar

English: Pattern matching includes expressions that can be used to filter data out, such as in the following example: **Türkçe:** Pattern matching, aşağıdaki örnekte olduğu gibi verileri filtrelemek için kullanılabilen expression'ları kapsar:

```
void printIntegersGreaterThan5(Number number) {
    if(number instanceof Integer data && data.compareTo(5)>0)
        System.out.print(data);
}
```

English: We can apply a number of filters, or patterns, so that the if statement is executed only in specific circumstances. Notice that we're using the pattern variable in an expression in the same line in which it is declared. **Türkçe:** `if` statement'ının yalnızca belirli koşullarda çalışması için birden çok filtre ya da pattern uygulayabiliriz. Pattern variable'ı bildirildiği satırdaki bir expression içinde kullandığımıza dikkat edin.

English: Subtypes The type of the pattern variable must be a subtype of the variable on the left side of the expression. It also cannot be the same type. This rule does not exist for traditional instanceof operator expressions, though. Consider the following two uses of the instanceof operator: **Türkçe:** Subtype'lar: Pattern variable'ın type'ı, expression'ın solundaki variable type'ının subtype'ı olmalıdır; aynı type olamaz. Buna karşılık bu kural geleneksel `instanceof` operator expression'ları için geçerli değildir. `instanceof` operator'ünün şu iki kullanımını düşünün:

```
Integer value = 123;
if(value instanceof Integer) {}
if(value instanceof Integer data) {} // DOES NOT COMPILE
```

English: While the second line compiles, the last line does not compile because pattern matching requires that the pattern variable type Integer be a strict subtype of Integer. **Türkçe:** İkinci satır derlenir; son satır ise derlenmez. Çünkü pattern matching, pattern variable type'ı `Integer`ın soldaki `Integer` type'ının strict subtype'ı olmasını gerektirir.

Limitations of Subtype Enforcement

Türkçe başlık: Subtype Denetiminin Sınırları

English: The compiler has some limitations on enforcing pattern matching types when we mix classes and interfaces, which will make more sense after you read Chapter 7, "Beyond Classes." For example, given the non-final class Number and interface List, this does compile even though they are unrelated: **Türkçe:** Class'lar ile interface'leri birlikte kullandığımızda derleyicinin pattern matching type kurallarını uygulamasında bazı sınırlar vardır; Bölüm 7, "Beyond Classes"ı okuduktan sonra bu konu daha anlamlı olacaktır. Örneğin final olmayan `Number` class'ı ile `List` interface'i birbiriyle ilişkili olmasa da aşağıdaki kod derlenir:

```
Number value = 123;
if(value instanceof List) {}
if(value instanceof List data) {}
```

Flow Scoping

Türkçe başlık: Flow Scoping

English: The compiler applies flow scoping when working with pattern matching. Flow scoping means the variable is only in scope when the compiler can definitively determine its type. **Türkçe:** Compiler pattern matching ile çalışırken flow scoping uygular. Flow scoping, variable'ın yalnızca compiler type'ını kesin olarak belirleyebildiğinde scope içinde olması demektir.

English: Flow scoping is unlike any other type of scoping in that it is not strictly hierarchical like instance, class, or local scoping. It is determined by the compiler based on the branching and flow of the program.

Türkçe: Flow scoping; instance scope, class scope veya local scope gibi katı bir hiyerarşiye dayanmadığı için diğer scope türlerinden ayrılır. Programın dallanmasına ve control flow'una göre compiler tarafından belirlenir.

English: Given this information, can you see why the following does not compile? **Türkçe:** Bu bilgi göz önüne alındığında, aşağıdakilerin neden derlenmediğini görebiliyor musunuz?

```
void printIntegersOrNumbersGreaterThan5(Number number) {
    if(number instanceof Integer data || data.compareTo(5)>0)
        System.out.print(data);
}
```

English: If the input does not inherit Integer, the data variable is undefined. Since the compiler cannot guarantee that data is an instance of Integer, data is not in scope, and the code does not compile. **Türkçe:** Input *Integer* dan miras almıyorsa *data* variable'ı tanımsızdır. Compiler, *data* nın bir *Integer* instance'ı olduğunu garanti edemediği için *data* scope içinde değildir ve kod derlenmez.

English: What about this example? **Türkçe:** Peki ya bu örnek?

```
void printIntegerTwice(Number number) {
    if (number instanceof Integer data)
        System.out.print(data.intValue());
    System.out.print(data.intValue()); // DOES NOT COMPILE
}
```

English: Since the input might not have inherited Integer, data is no longer in scope after the if statement. Oh, so you might be thinking that the pattern variable is then only in scope inside the if statement, right? Well, not exactly! Consider the following example that does compile: **Türkçe:** Input *Integer* dan miras almıyor olabileceği için *data*, *if* statement'ından sonra artık scope içinde değildir. Buradan pattern variable'ın yalnızca *if* statement'ının scope'unda olduğunu düşünebilirsiniz; ancak tam olarak öyle değildir. Derlenen şu örneği inceleyin:

```
void printOnlyIntegers(Number number) {  
    if (!(number instanceof Integer data))  
        return;  
    System.out.print(data.intValue());  
}
```

English: It might surprise you to learn this code does compile. Eek! What is going on here? The method returns if the input does not inherit Integer. This means that when the last line of the method is reached, the input must inherit Integer, and therefore data stays in scope even after the if statement ends. **Türkçe:** Bu kodun derlenmesi şaşırtıcı gelebilir. Burada ne oluyor? Input `Integer`dan miras almıyorsa method `return` ile sona erer. Dolayısıyla method'un son satırına ulaşılmışsa input kesinlikle `Integer`dır ve `data`, `if` statement'ı bittikten sonra da scope içinde kalır.

Flow Scoping and else Branches

Türkçe başlık: Flow Scoping ve `else` Dalları

English: If the last code sample confuses you, don't worry: you're not alone! Another way to think about it is to rewrite the logic to something equivalent that uses an else statement: **Türkçe:** Son kod örneği kafanızı karıştırdıysa endişelenmeyin: yalnız değilsiniz! Bunu düşünmenin başka bir yolu da, mantığı else ifadesini kullanan eşdeğer bir şeye yeniden yazmaktır:

```
void printOnlyIntegers(Number number) {  
    if (!(number instanceof Integer data))  
        return;  
    System.out.print(data.intValue());  
}
```

English: else **Türkçe:** `else`

```
System.out.print(data.intValue());  
}
```

English: We can now go one step further and reverse the if and else branches by inverting the boolean expression: **Türkçe:** Artık bir adım daha ileri gidebilir ve boolean expression'ı ters çevirerek if ve else dallarını tersine çevirebiliriz:

```
void printOnlyIntegers(Number number) {  
    if (number instanceof Integer data)
```

```
        System.out.print(data.intValue());
```

English: else **Türkçe:** `else`

```
        return;  
    }
```

English: Our new code is equivalent to our original and better demonstrates how the compiler was able to determine that data was in scope only when number is an Integer. **Türkçe:** Yeni kodumuz ilk kodla eşdeğerdir ve compiler'ın `data`nın yalnızca `number` bir `Integer` olduğunda scope içinde bulunduğunu nasıl belirlediğini daha açık gösterir.

English: Make sure you understand the way flow scoping works. In particular, it is possible to use a pattern variable outside of the if statement, but only when the compiler can definitively determine its type. **Türkçe:** Flow scoping'in nasıl çalıştığını anladığınızdan emin olun. Bir pattern variable `if` statement'ının dışında kullanılabilir; ancak yalnızca compiler onun type'ını kesin olarak belirleyebiliyorsa.

Applying switch Statements

Türkçe başlık: `switch` Statement ve Expression'larını Uygulama

English: What if we have a lot of possible branches or paths for a single value? For example, we might want to print a different message based on the day of the week. We could certainly accomplish this with a combination of seven if or else statements, but that tends to create code that is long, difficult to read, and often not fun to maintain. **Türkçe:** Tek bir değer için çok sayıda olası branch veya execution path varsa ne olur? Örneğin haftanın gününe göre farklı bir mesaj yazdırmak isteyebiliriz. Bunu yedi `if/else` statement'ını birleştirerek yapabiliriz; ancak ortaya çoğunlukla uzun, okunması zor ve bakımı zahmetli bir kod çıkar:

```
public void printDayOfWeek(int day) {  
    if(day == 0)  
        System.out.print("Sunday");  
    else if(day == 1)  
        System.out.print("Monday");  
    else if(day == 2)  
        System.out.print("Tuesday");  
    else if(day == 3)  
        System.out.print("Wednesday");  
}
```

English: ... **Türkçe:** ...

```
}
```

English: Luckily, Java, along with many other languages, provides a cleaner approach. In this section we present the switch statement, along with the newer switch expression for controlling program flow. **Türkçe:** Neyse ki Java, diğer pek çok dil gibi daha temiz bir yaklaşım sunar. Bu bölümde program akışını kontrol eden `switch` statement'ını ve daha yeni `switch` expression'ını tanıtıyoruz.

The switch Statement

Türkçe başlık: `switch` Statement'i

English: A switch statement, as shown in Figure 3.3, is a complex decision-making structure in which a single value is evaluated and flow is redirected to the first matching branch, known as a case statement. If no such case statement is found that matches the value, an optional **Türkçe:** Şekil 3.3'te gösterildiği gibi `switch` statement'ı, tek bir değeri değerlendirip akışı `case` statement'ı denen ilk eşleşen dala yönlendiren karmaşık bir decision-making yapısıdır. Değerle eşleşen bir `case` statement'ı bulunmazsa isteğe bağlı bir

English: default statement will be called. If no such default option is available, the entire switch statement will be skipped. Notice in Figure 3.3 that case values can be combined into a single case statement using commas. **Türkçe:** `default` statement'ı çağrılır. Böyle bir `default` seçeneği de yoksa `switch` statement'ının tamamı atlanır. Şekil 3.3'te `case` değerlerinin virgülle tek bir `case` statement'ında birleştirilebildiğine dikkat edin.

FIGURE 3.3 The structure of a switch statement

Türkçe başlık: ŞEKİL 3.3 Bir `switch` statement'ının yapısı

English: switch keyword Parentheses (required) **Türkçe:** `switch` keyword'ü; parantez zorunludur.

```
switch(variableToTest) {
```

English: Beginning curly brace (required) **Türkçe:** Başlangıç küme ayracı (gerekli)

```
case constantExpression1:
// Branch for case1
break;
```

English: case constantExpression2, constantExpression3 **Türkçe:** `case constantExpression2, constantExpression3`

```
// Branch for case2 and case3
break;
```

Optional break

Türkçe başlık: İsteğe Bağlı `break`

English: ... **Türkçe:** ...

English: Optional default that may appear anywhere within switch statement **Türkçe:** `switch` statement'ının herhangi bir yerinde bulunabilen isteğe bağlı `default`

```
}
default:
// Branch for default
```

English: Ending curly brace (required) Because switch statements can be longer than most decision-making statements, the exam may present invalid switch syntax to see whether you are paying attention. **Türkçe:** Küme ayracı sonlandırma (gerekli) `switch` statement'ları çoğu karar verme ifadesinden daha uzun olabileceğinden, sınav, dikkat edip etmediğinizi görmek için geçersiz switch söz dizimi sunabilir.

English: Combining case Values Notice something new in Figure 3.3? Starting with Java 14, case values can now be combined: **Türkçe:** `case` Değerlerini Birleştirme: Şekil 3.3'te yeni bir şey fark ettiniz mi? Java 14'ten itibaren `case` değerleri birleştirilebilir:

```
switch (animal) {  
  case 1,2: System.out.print("Lion");  
  case 3: System.out.print("Tiger");  
}
```

English: Prior to Java 14, the equivalent code would have been the following: **Türkçe:** Java 14'ten önce eşdeğer kod aşağıdaki gibi olurdu:

```
switch (animal) {  
  case 1: case 2: System.out.print("Lion");  
  case 3: System.out.print("Tiger");  
}
```

English: As you see shortly, switch expressions can reduce boilerplate code even more! **Türkçe:** Birazdan göreceğiniz gibi `switch` expression'ları boilerplate code'u daha da azaltabilir.

English: See if you can figure out why each of the following switch statements does not compile: **Türkçe:** Aşağıdaki `switch` statement'larının her birinin neden derlenmediğini bulmaya çalışın:

```
int month = 5;  
switch month { // DOES NOT COMPILE  
  case 1: System.out.print("January");  
}  
switch (month) // DOES NOT COMPILE  
  case 1: System.out.print("January");  
  switch (month) {  
  case 1: 2: System.out.print("January"); // DOES NOT COMPILE  
}
```

English: The first switch statement does not compile because it is missing parentheses around the switch variable. The second statement does not compile because it is missing braces around the switch body. The third statement does not compile because a comma (,) should be used to separate combined case statements, not a colon (:). **Türkçe:** İlk `switch` statement'ı, `switch` variable'ını çevreleyen parantezler eksik olduğu için derlenmez. İkinci statement, `switch` gövdesini çevreleyen küme parantezleri eksik olduğu için derlenmez. Üçüncü statement ise birleştirilmiş `case` değerlerini ayırmak için iki nokta (:) yerine virgül (,) kullanılması gerektiğinden derlenmez.

English: One last note you should be aware of for the exam: a switch statement is not required to contain any case statements. For example, this statement is perfectly valid: **Türkçe:** Sınav için bilmeniz gereken son bir nokta şudur: Bir `switch` statement'ının `case` statement'ı içermesi zorunlu değildir. Örneğin aşağıdaki statement tamamen geçerlidir:

```
switch(month) {}
```

English: Going back to our `printDayOfWeek()` method, we can rewrite it to use a switch statement instead of if/else statements: **Türkçe:** `printDayOfWeek()` method'una dönersek onu `if/else` statement'ları yerine `switch` statement'ı kullanan biçimde yeniden yazabiliriz:

```
public void printDayOfWeek(int day) {  
    switch(day) {  
        case 0:  
            System.out.print("Sunday");  
            break;  
        case 1:  
            System.out.print("Monday");  
            break;  
        case 2:  
            System.out.print("Tuesday");  
            break;
```

```
        case 3:  
            System.out.print("Wednesday");  
            break;  
        case 4:  
            System.out.print("Thursday");  
            break;  
        case 5:  
            System.out.print("Friday");  
            break;  
        case 6:  
            System.out.print("Saturday");  
            break;  
        default:  
            System.out.print("Invalid value");  
            break;  
    } }  
}
```

English: For simplicity, we just print a message if the value is invalid. If you know about exceptions or have already read Chapter 11, “Exceptions and Localization,” it might make more sense to throw an exception in the default branch if no match is found. **Türkçe:** Basitlik açısından, değer geçersizse bir mesaj yazdırırız. İstisnalar hakkında bilginiz varsa veya Bölüm 11, “İstisnalar ve Yerelleştirme”yi zaten okuduysanız, eşleşme bulunmazsa default dalına bir istisna atmak daha mantıklı olabilir.

Exiting with break Statements

Türkçe başlık: `break` Statement'larıyla Çıkış

English: Taking a look at our previous `printDayOfWeek()` implementation, you'll see a `break` statement at the end of each case and default section. A `break` statement terminates the switch statement and returns flow control to the enclosing process. Put simply, it ends the switch statement immediately. **Türkçe:** Önceki `printDayOfWeek()` implementasyonunda her `case` ve `default` bölümünün sonunda bir `break` statement'ı bulunur. `break`, `switch` statement'ını sonlandırıp control flow'u enclosing process'e aktarır; başka bir deyişle `switch` statement'ından hemen çıkar.

English: The `break` statements are optional, but without them the code will execute every branch following a matching case statement, including any default statements it finds. Without `break` statements in each branch, the order of case and default statements is now extremely important. What do you think the following prints when `printSeason(2)` is called? **Türkçe:** `break` statement'ları isteğe bağlıdır; ancak bunlar olmadan kod, karşılaştığı `default` dahil eşleşen `case` statement'ından sonraki bütün branch'leri çalıştırır. Her branch'te `break` bulunmadığında `case` ve `default` sırası son derece önemlidir. `printSeason(2)` çağrıldığında aşağıdakilerden hangisi yazdırılır?

```
public void printSeason(int month) {  
    switch(month) {  
        case 1, 2, 3: System.out.print("Winter");  
        case 4, 5, 6: System.out.print("Spring");  
        default: System.out.print("Unknown");  
        case 7, 8, 9: System.out.print("Summer");  
        case 10, 11, 12: System.out.print("Fall");  
    }  
}
```

English: It prints everything! **Türkçe:** Her şeyi yazdırıyor!

English: WinterSpringUnknownSummerFall **Türkçe:** WinterSpringUnknownSummerFall

English: It matches the first case statement and executes all of the branches in the order they are found, including the default statement. It is common, although certainly not required, to use a `break` statement after every case statement. **Türkçe:** Kod ilk `case` statement'ıyla eşleşir ve `default` statement dahil sonraki bütün branch'leri sırasıyla çalıştırır. Zorunlu olmasa da her `case` statement'ından sonra bir `break` statement'ı kullanmak yaygındır.

English: The exam creators are fond of switch examples that are missing `break` statements! When evaluating switch statements on the exam, always consider that multiple branches may be visited in a single execution. **Türkçe:** Sınav yazarları `break` statement'ları eksik `switch` örneklerini sever. Sınavda bir `switch` statement'ını değerlendirirken tek bir yürütmeye birden çok dala girilebileceğini her zaman hesaba katın.

Selecting switch Data Types

Türkçe başlık: `switch` Veri Türlerini Seçme

English: As shown in Figure 3.3, a switch statement has a target variable that is not evaluated until runtime. The type of this target can include select primitive data types (int, byte, short, char) and their associated wrapper classes (Integer, Byte, Short, Character). The following is a list of all data types supported by switch statements: **Türkçe:** Şekil 3.3'te gösterildiği gibi bir `switch` statement'ının runtime'a kadar değerlendirilmeyen bir target variable'ı vardır. Bu target'ın type'ı belirli primitive data type'ları (`int`, `byte`, `short`, `char`) ve bunların wrapper class'larını (`Integer`, `Byte`, `Short`, `Character`) içerebilir. `switch` statement'larının desteklediği bütün data type'lar şunlardır:

English: ■ ■ int and Integer **Türkçe:** ■ ■ int ve Integer

English: ■ ■ byte and Byte **Türkçe:** ■ ■ `byte` ve `Byte`

English: ■ ■ short and Short **Türkçe:** ■ ■ `short` ve `Short`

English: ■ ■ char and Character **Türkçe:** ■ ■ `char` ve `Character`

English: ■ ■ String **Türkçe:** ■ ■ `String`

English: ■ ■ enum values **Türkçe:** ■ ■ enum değerleri

English: ■ ■ var (if the type resolves to one of the preceding types) For this chapter, you just need to know that an enumeration, or enum, represents a fixed set of constants, such as days of the week, months of the year, and so on. We cover enums in more detail in Chapter 7, including showing how they can define variables, methods, and constructors. **Türkçe:** ■ ■ `var` (type önceki type'lardan birine çözümleniyorsa) Bu bölümde enumeration'ın, yani enum'un haftanın günleri ve yılın ayları gibi sabit bir constant kümesini temsil ettiğini bilmeniz yeterlidir. Variable, method ve constructor tanımlayabilmeleri dahil enum'ları Bölüm 7'de daha ayrıntılı ele alıyoruz.

English: Notice that boolean, long, float, and double are excluded from switch statements, as are their associated Boolean, Long, Float, and Double classes. The reasons are varied, such as boolean having too small a range of values and floating-point numbers having quite a wide range of values. For the exam, though, you just need to know that they are not permitted in switch statements. **Türkçe:** `boolean`, `long`, `float` ve `double` ile bunlara karşılık gelen `Boolean`, `Long`, `Float` ve `Double` class'larının `switch` statement'larında kullanılamadığına dikkat edin. Nedenler çeşitlidir: örneğin `boolean`'ın değer aralığı çok küçük, floating-point sayılarınki ise oldukça geniştir. Sınav için bunlara `switch` statement'larında izin verilmediğini bilmeniz yeterlidir.

Determining Acceptable Case Values

Türkçe başlık: Kabul Edilebilir `case` Değerlerinin Belirlenmesi

English: Not just any variable or value can be used in a case statement. First, the values in each case statement must be compile-time constant values of the same data type as the switch value. This means you can use only literals, enum constants, or final constant variables of the same data type. By final constant, we mean that the variable must be marked with the final modifier and initialized with a literal value in the same expression in which it is declared. For example, you can't have a case statement value that requires executing a

Türkçe: Bir `case` statement'ında her variable veya değer kullanılamaz. Öncelikle her `case` statement'ındaki değer, `switch` değeriyle aynı data type'ta bir compile-time constant olmalıdır. Yalnız aynı data type'taki literal'lar, enum constant'ları veya final constant variable'lar kullanılabilir. Final constant ile variable'ın `final` modifier'ıyla işaretlenmesini ve bildirildiği expression içinde bir literal ile initialize edilmesini kastediyoruz. Örneğin bir `case` değeri,

English: method at runtime, even if that method always returns the same value. For these reasons, only the first and last case statements in the following example compile: **Türkçe:** method'un runtime'da çalıştırılmasını gerektiren bir değer kullanamazsınız; method her zaman aynı değeri döndürse bile. Bu nedenle aşağıdaki örnekte yalnızca ilk ve son `case` statement'ı derlenir:

```
final int getCookies() { return 4; }
void feedAnimals() {
    final int bananas = 1;
    int apples = 2;
    int numberOfAnimals = 3;
    final int cookies = getCookies();
    switch(numberOfAnimals) {
        case bananas:
        case apples: // DOES NOT COMPILE
        case getCookies(): // DOES NOT COMPILE
        case cookies : // DOES NOT COMPILE
        case 3 * 5 :
    } }
```

English: The bananas variable is marked final, and its value is known at compile-time, so it is valid. The apples variable is not marked final, even though its value is known, so it is not permitted. The next two case statements, with values `getCookies()` and `cookies`, do not compile because methods are not evaluated until runtime, so they cannot be used as the value of a case statement, even if one of the values is stored in a final variable. The last case statement, with value `3 * 5`, does compile, as expressions are allowed as case values, provided the value can be resolved at compile-time. They also must be able to fit in the switch data type without an explicit cast. We go into that in more detail shortly. **Türkçe:** `bananas` variable'ı `final`dır ve değeri compile-time'da bilindiği için geçerlidir. `apples` variable'ı ise değeri bilinse de `final` olmadığı için kullanılamaz. `getCookies()` ve `cookies` değerli sonraki iki `case` statement'ı derlenmez; method'lar runtime'a kadar değerlendirilmediğinden, sonuç final variable'da saklansa bile `case` değeri olamaz. `3 * 5` değerli son `case` statement'ı derlenir; çünkü expression compile-time'da çözümlenebiliyorsa `case` değeri olabilir. Ayrıca açık cast olmadan `switch` data type'ına sığmalıdır. Bunu birazdan daha ayrıntılı ele alacağız.

English: Next, the data type for case statements must match the data type of the switch variable. **Türkçe:** Ayrıca `case` statement'larının data type'ı, `switch` variable'ının data type'ıyla eşleşmelidir.

English: For example, you can't have a case statement of type `String` if the switch statement variable is of type `int`, since the types are incomparable. **Türkçe:** Örneğin `switch` expression variable'ının type'ı `int` ise type'lar karşılaştırılmadığı için `String` type'ında bir `case` statement kullanamazsınız.

The switch Expression

Türkçe başlık: `switch` Expression'i

English: Our second implementation of `printDayOfWeek()` was improved but still quite long. Notice that there was a lot of boilerplate code, along with numerous `break` statements. Can we do better? Yes, thanks to the new switch expressions that were officially added to Java 14. **Türkçe:** `printDayOfWeek()` method'unun ikinci implementasyonu daha iyi olsa da hâlâ uzundu. Çok sayıda `break` statement'ıyla birlikte epey boilerplate code bulunduğuna dikkat edin. Java 14'e resmen eklenen `switch` expression'ları sayesinde bunu daha iyi yapabiliriz.

English: A switch expression is a much more compact form of a switch statement, capable of returning a value. Take a look at the new syntax in Figure 3.4. **Türkçe:** `switch` expression, `switch` statement'inin değer döndürebilen çok daha kompakt bir biçimidir. Şekil 3.4'teki yeni syntax'a bakın.

English: Because a switch expression is a compact form, there's a lot going on in Figure 3.4! **Türkçe:** `switch` expression kompakt bir yapı olduğundan Şekil 3.4'te aynı anda pek çok unsur gösterilir.

English: For starters, we can now assign the result of a switch expression to a variable result. For this to work, all case and default branches must return a data type that is compatible with the assignment. The switch expression supports two types of branches: an expression and a block. Each has different syntactical rules on how it must be created. More on these topics shortly. **Türkçe:** Öncelikle bir `switch` expression'ının sonucunu artık `result` variable'ına atayabiliriz. Bunun çalışması için bütün `case` ve `default` branch'leri assignment ile uyumlu bir data type döndürmelidir. `switch` expression iki tür branch destekler: expression ve block. Her birinin farklı syntax kuralları vardır; bunları birazdan ele alacağız.

FIGURE 3.4 The structure of a switch expression

Türkçe başlık: ŞEKİL 3.4 Bir `switch` expression'ın yapısı

English: `switch` keyword Optional assignment Parentheses (required) **Türkçe:** `switch` keyword'ü · İsteğe bağlı assignment · Parantezler zorunludur

```
int result = switch(variableToTest) {
```

English: Beginning curly brace (required) Arrow operator (required) case expression case block **Türkçe:** Başlangıç küme parantezi zorunludur · Arrow operator (`->`) zorunludur · `case` expression · `case` block

```
case constantExpression1-> 5;
```

English: Semicolon required for case expression **Türkçe:** `case` expression için noktalı virgül zorunludur

```
case constantExpression2, constantExpression3 -> {  
    yield 10;  
}
```

English: Required for case block if switch returns a value Curly braces required for case blocks ...

Türkçe: `switch` bir değer döndürüyorsa `case` block için `yield` zorunludur · `case` block'larında küme parantezleri zorunludur · ...

English: A default branch may appear anywhere within the switch expression and is required if all possible case statement values are not handled. **Türkçe:** Bir `default` branch `switch` expression'ın herhangi bir yerinde bulunabilir; bütün olası `case` değerleri işlenmiyorsa zorunludur.

```
}  
default -> 20;
```

English: Ending curly brace (required) Like a traditional switch statement, a switch expression supports zero or many case branches and an optional default branch. Both also support the new feature that allows case values to be combined with a single case statement using commas. Unlike a traditional switch statement, though, switch expressions have special rules around when the default branch is required.

Türkçe: Bitiş küme parantezi zorunludur. Geleneksel `switch` statement'ı gibi `switch` expression da sıfır veya daha çok `case` branch'i ile isteğe bağlı bir `default` branch destekler. Her ikisinde de birden çok `case` değeri virgülle tek bir `case` statement'ında birleştirilebilir. Ancak geleneksel `switch` statement'ından farklı olarak `switch` expression'da `default` branch'in ne zaman zorunlu olduğuna ilişkin özel kurallar vardır.

English: Recall from Chapter 2, "Operators," that `->` is the arrow operator. While the arrow operator is commonly used in lambda expressions, when it is used in a switch expression, the case branches are not lambdas. **Türkçe:** Bölüm 2, "Operators"dan hatırlayın: `->` arrow operator'dür. Arrow operator lambda expression'larda yaygın kullanılsa da `switch` expression içindeki `case` branch'leri lambda değildir.

English: We can rewrite our previous `printDayOfWeek()` method in a much more concise manner using case expressions: **Türkçe:** Önceki `printDayOfWeek()` method'umuzu `case` expression'larıyla çok daha kısa biçimde yeniden yazabiliriz:

```
public void printDayOfWeek(int day) {  
    var result = switch(day) {  
        case 0 -> "Sunday";  
        case 1 -> "Monday";  
        case 2 -> "Tuesday";  
        case 3 -> "Wednesday";  
        case 4 -> "Thursday";  
        case 5 -> "Friday";  
        case 6 -> "Saturday";
```

```
        default -> "Invalid value";  
    };  
    System.out.print(result);  
}
```

English: Compare this code with the switch statement we wrote earlier. Both accomplish the same task, but a lot of the boilerplate code has been removed, leaving the behavior we care most about. **Türkçe:** Bu kodu daha önce yazdığımız `switch` statement'ıyla karşılaştırın. İkisi de aynı işi yapar; ancak boilerplate code'un çoğu kaldırılmış, geriye asıl önemsedığımız davranış kalmıştır.

English: Notice that a semicolon is required after each switch expression. For example, the following code does not compile. How many semicolons is it missing? **Türkçe:** Her `switch` expression'ından sonra noktalı virgül gerektiğine dikkat edin. Örneğin aşağıdaki kod derlenmez. Kaç noktalı virgül eksiktir?

```
var result = switch(bear) {  
  case 30 -> "Grizzly"  
  default -> "Panda"  
}
```

English: The answer is three. Each case or default expression requires a semicolon as well as the assignment itself. The following fixes the code: **Türkçe:** Cevap üçtür. Her `case` veya `default` expression'ın ve assignment'ın sonunda birer noktalı virgül gerekir. Aşağıdaki kod hataları düzeltir:

```
var result = switch(bear) {  
  case 30 -> "Grizzly";  
  default -> "Panda";  
};
```

English: As shown in Figure 3.4, case statements can take multiple values, separated by commas. **Türkçe:** Şekil 3.4'te gösterildiği gibi, `case` dalları virgülle ayrılmış birden fazla değer alabilir.

English: Let's rewrite our `printSeason()` method from earlier using a switch expression: **Türkçe:** Daha önce kullandığımız `printSeason()` method'unu bir `switch` expression'ı kullanarak yeniden yazalım:

```
public void printSeason(int month) {  
  switch(month) {  
  case 1, 2, 3 -> System.out.print("Winter");  
  case 4, 5, 6 -> System.out.print("Spring");  
  case 7, 8, 9 -> System.out.print("Summer");  
  case 10, 11, 12 -> System.out.print("Fall");  
  } }  
}
```

English: Calling `printSeason(2)` prints the single value Winter. This time we don't have to worry about break statements, since only one branch is executed. **Türkçe:** `printSeason(2)` çağrısı yalnızca `Winter` değerini yazdırır. Bu kez yalnızca bir branch çalıştığından `break` statement'lariyle ilgilenmemiz gerekmez.

English: Most of the time, a switch expression returns a value, although `printSeason()` demonstrates one in which the return type is void. **Türkçe:** Çoğu zaman, bir `switch` expression'ı bir değer döndürür, ancak `printSeason()` method'u dönüş türünün `void` olduğu bir ifadeyi gösterir.

English: Since the type is void, it can't be assigned to a variable. On the exam, you are more likely to see a switch expression that returns a value, but you should be aware that it is possible. **Türkçe:** `void` türü olduğundan bir değişkene atanamaz. Sınavda değer döndüren bir `switch` expression'ı görme olasılığınız daha yüksektir ancak bunun mümkün olduğunu bilmelisiniz.

English: All of the previous rules around switch data types and case values still apply, although we have some new rules. Don't worry if these rules are new to you or you've never seen the `yield` keyword before; we'll be discussing them in the following sections. **Türkçe:** Bazı yeni kurallar eklenmiş olsa da `switch` data type'ları ve `case` değerleriyle ilgili önceki kuralların tümü geçerliliğini korur. Bu kurallar sizin için yeniyse veya `yield` keyword'ünü daha önce hiç görmediyseniz endişelenmeyin; bunları ilerleyen bölümlerde ele alacağız.

English: 1. All of the branches of a switch expression that do not throw an exception must return a consistent data type (if the switch expression returns a value). **Türkçe:** 1. Exception fırlatmayan bütün `switch` expression branch'leri tutarlı bir data type döndürmelidir (`switch` expression bir değer döndürüyorsa).

English: 2. If the switch expression returns a value, then every branch that isn't an expression must yield a value. **Türkçe:** 2. `switch` expression'ı bir değer döndürüyorsa, ifade olmayan her dalın bir değer vermesi gerekir.

English: 3. A default branch is required unless all cases are covered or no value is returned. **Türkçe:** 3. Tüm durumlar kapsamadığı veya herhangi bir değer döndürülmediği sürece default dallanma gereklidir.

English: We cover the last rule shortly, but notice that our `printSeason()` example does not contain a default branch. Since the switch expression does not return a value and assign it to a variable, it is entirely optional. **Türkçe:** Son kuralı kısaca ele alacağız, ancak `printSeason()` örneğimizin bir default dal içermediğine dikkat edin. `switch` expression'ı bir değer döndürmediği ve bunu bir değişkene atamadığı için tamamen isteğe bağlıdır.

English: Java 17 also supports pattern matching within switch expressions, but since this is a Preview feature, it is not in scope for the exam. **Türkçe:** Java 17 aynı zamanda `switch` ifadeleri içindeki pattern matching'i de destekler, ancak bu bir Önizleme özelliği olduğundan sınavın kapsamında değildir.

Returning Consistent Data Types

Türkçe başlık: Tutarlı Veri Türlerini Döndürme

English: The first rule of using a switch expression is probably the easiest. You can't return incompatible or random data types. For example, can you see why three of the lines of the following code do not compile? **Türkçe:** `switch` expression kullanmanın ilk kuralı muhtemelen en kolaydır: Birbiriyle uyumsuz veya rastgele data type'lar döndüremezsiniz. Aşağıdaki kodda üç satırın neden derlenmediğini görebiliyor musunuz?

```
int measurement = 10;
int size = switch(measurement) {
  case 5 -> 1;
  case 10 -> (short)2;
  default -> 5;
  case 20 -> "3"; // DOES NOT COMPILE
  case 40 -> 4L; // DOES NOT COMPILE
  case 50 -> null; // DOES NOT COMPILE
};
```

English: Notice that the second case expression returns a short, but that can be implicitly cast to an int. In this manner, the values have to be consistent with size, but they do not all have to be the same data type. The last three case expressions do not compile because each returns a type that cannot be assigned to the int variable. **Türkçe:** İkinci case statement'ın short döndürdüğüne dikkat edin; bu değer implicitly inte dönüştürülebilir. Dolayısıyla değerlerin boyut açısından uyumlu olması gerekir, fakat hepsinin aynı data type'ta olması şart değildir. Son üç case branch'i ise int variable'a atanamayan bir type döndürdüğü için derlenmez.

English: Applying a case Block A switch expression supports both an expression and a block in the case and default branches. Like a regular block, a case block is one that is surrounded by braces {}. It also includes a yield statement if the switch expression returns a value. For example, the following uses a mix of case expressions and blocks: **Türkçe:** Bir case Bloğu uygulama switch expression'ı, case ve default dallarındaki hem bir ifadeyi hem de bir bloğu destekler. Normal bir blok gibi, case bloğu da parantezlerle {} çevrelenen bir bloktur. Ayrıca, switch expression'ı bir değer döndürüyorsa, bir yield ifadesini de içerir. Örneğin, aşağıdakiler case ifadeleri ve bloklarının bir karışımını kullanır:

```
int fish = 5;
int length = 12;
var name = switch(fish) {
  case 1 -> "Goldfish";
  case 2 -> {yield "Trout";}
  case 3 -> {
    if(length > 10) yield "Blobfish";
    else yield "Green";
  }
  default -> "Swordfish";
};
```

English: The yield keyword is equivalent to a return statement within a switch expression and is used to avoid ambiguity about whether you meant to exit the block or method around the switch expression.

Türkçe: yield keyword'ü, switch expression içindeki return statement'ına denktir. switch expression'ı çevreleyen block'tan mı yoksa method'dan mı çıkılacağına ilişkin belirsizliği önler.

English: Referring to our second rule for switch expressions, yield statements are not optional if the switch statement returns a value. Can you see why the following lines do not compile? **Türkçe:** switch expression'lara ilişkin ikinci kurala göre, switch bir değer döndürüyorsa yield statement'ları isteğe bağlı değildir. Aşağıdaki satırların neden derlenmediğini görebiliyor musunuz?

```
10: int fish = 5;
11: int length = 12;
12: var name = switch(fish) {
13: case 1 -> "Goldfish";
14: case 2 -> {} // DOES NOT COMPILE
15: case 3 -> {
16: if(length > 10) yield "Blobfish";
17: } // DOES NOT COMPILE
18: default -> "Swordfish";
19: };
```

English: Line 14 does not compile because it does not return a value using yield. Line 17 also does not compile. While the code returns a value for length greater than 10, it does not return a value if length is less than or equal to 10. It does not matter that length is set to be 12; all branches must yield a value within the case block. **Türkçe:** 14. satır `yield` ile değer üretmediği için derlenmez. 17. satır da derlenmez. Kod `length` 10'dan büyükken değer üretir; fakat 10'a eşit veya daha küçükken üretmez. `length`in 12 olarak atanması önemli değildir; `case` block'undaki bütün path'ler bir değer üretmelidir.

Watch Semicolons in switch Expressions

Türkçe başlık: `switch` Expression'larında Noktalı Virgüllere Dikkat

English: Unlike a regular switch statement, a switch expression can be used with the assignment operator and requires a semicolon when doing so. Furthermore, semicolons are required for case expressions but cannot be used with case blocks. **Türkçe:** Normal bir `switch` statement'ından farklı olarak `switch` expression assignment operator'üyle kullanılabilir ve bu kullanımda noktalı virgül gerekir. Ayrıca `case` expression'larda noktalı virgül zorunludur; `case` block'larında ise kullanılamaz.

```
var name = switch(fish) {
case 1 -> "Goldfish" // DOES NOT COMPILE (missing semicolon)
```

```
case 2 -> {yield "Trout";}; // DOES NOT COMPILE (extra semicolon)
```

English: ... **Türkçe:** ...

English: `} // DOES NOT COMPILE (missing semicolon)` A bit confusing, right? It's just one of those things you have to train yourself to spot on the exam. **Türkçe:** `} // DOES NOT COMPILE (semicolon eksik)` Biraz kafa karıştırıcı, değil mi? Bu, sınavda görmeye alışmanız gereken ayrıntılardan biridir.

Covering All Possible Values

Türkçe başlık: Bütün Olası Değerleri Kapsama

English: The last rule about switch expressions is probably the one the exam is most likely to try to trick you on: a switch expression that returns a value must handle all possible input values. **Türkçe:** `switch` expression'lariyle ilgili son kural muhtemelen sınavın sizi kandırmaya çalışacağı kuraldır: bir değer döndüren `switch` expression'ı tüm olası giriş değerlerini işlemelidir.

English: And as you saw earlier, when it does not return a value, it is optional. **Türkçe:** Ve daha önce gördüğünüz gibi, bir değer döndürmediğinde isteğe bağlıdır.

English: Let's try this out. Given the following code, what is the value of type if canis is 5? **Türkçe:** Hadi bunu deneyelim. Aşağıdaki kod göz önüne alındığında, canis 5 ise type değeri nedir?

```
String type = switch(canis) { // DOES NOT COMPILE
case 1 -> "dog";
case 2 -> "wolf";
case 3 -> "coyote";
};
```

English: There's no case branch to cover 5 (or 4, -1, 0, etc.), so should the switch expression return null, the empty string, undefined, or some other value? When adding switch expressions to the Java language, the authors decided this behavior would be unsupported. Every switch expression must handle all possible values of the switch variable. As a developer, there are two ways to address this: **Türkçe:** 5'i (veya 4, -1, 0 vb.) kapsayan bir case branch'i yoktur. Öyleyse switch expression null, empty String, undefined veya başka bir değer mi döndürmelidir? Java'ya switch expression'ları eklenirken bu davranışın desteklenmemesine karar verilmiştir. Her switch expression, switch variable'in bütün olası değerlerini işlemelidir. Bu sorunu çözmenin iki yolu vardır:

English: ■ ■ ■ ■ Add a default branch. If the switch expression takes an enum value, add a case branch for every possible enum value. **Türkçe:** ■ ■ ■ ■ Bir default dal ekleyin. switch expression'ı bir enum değeri alıyorsa, olası her enum değeri için bir case dalı ekleyin.

English: In practice, the first solution is the one most often used. The second solution applies only to switch expressions that take an enum. You can try writing case statements for all possible int values, but we promise it doesn't work! Even smaller types like byte are not permitted by the compiler, despite there being only 256 possible values. **Türkçe:** Uygulamada en sık ilk çözüm kullanılır. İkinci çözüm yalnızca enum alan switch expression'lar için geçerlidir. Bütün olası int değerleri için case statement'ı yazmayı deneyebilirsiniz; fakat bu çalışmaz. Yalnızca 256 olası değer bulunmasına rağmen byte gibi daha küçük type'lara bile compiler izin vermez.

English: For enums, the second solution works well when the number of enum values is relatively small. For example, consider the following enum definition and method: **Türkçe:** Enum'lar için ikinci çözüm, enum değerlerinin sayısı nispeten küçük olduğunda işe yarar. Örneğin, aşağıdaki enum tanımını ve method'unu göz önünde bulundurun:

```
enum Season {WINTER, SPRING, SUMMER, FALL}
String getWeather(Season value) {
return switch(value) {
case WINTER -> "Cold";
case SPRING -> "Rainy";
case SUMMER -> "Hot";
case FALL -> "Warm";
};
}
```

English: Since all possible permutations of Season are covered, a default branch is not required in this switch expression. You can include an optional default branch, though, even if you cover all known values.

Türkçe: Season'ın bütün olası değerleri kapsandığı için bu switch expression'da default branch zorunlu değildir. Yine de bilinen bütün değerleri kapsasanız bile isteğe bağlı bir default branch ekleyebilirsiniz.

English: What happens if you use an enum with three values and later someone adds a fourth value? Any switch expressions that use the enum without a default branch will suddenly fail to compile. If this was done frequently, you might have a lot of code to fix! For this reason, consider including a default branch in every switch expression, even those that involve enum values.

Türkçe: Üç değeri olan bir enum kullanırken sonradan dördüncü bir enum constant eklenirse ne olur? default branch'i bulunmadan bu enum'u kullanan her switch expression artık derlenmez. Bu kullanım yaygınsa düzeltmeniz gereken çok sayıda kod olabilir. Bu nedenle enum değerlerini kullananlar dahil her switch expression'a bir default branch eklemeyi düşünün.

Writing while Loops

Türkçe başlık: while Loop'larını Yazma

English: A common practice when writing software is doing the same task some number of times.

Türkçe: Yazılım yazarken yaygın bir uygulama, aynı görevi birkaç kez yapmaktır.

English: You could use the decision structures we have presented so far to accomplish this, but that's going to be a pretty long chain of if or else statements, especially if you have to execute the same thing 100 times or more. **Türkçe:** Bunu başarmak için şu ana kadar sunduğumuz karar yapılarını kullanabilirsiniz, ancak bu, özellikle aynı şeyi 100 veya daha fazla kez yürütmeniz gerekiyorsa, oldukça uzun bir if veya else ifadeleri zinciri olacaktır.

English: Enter loops! A loop is a repetitive control structure that can execute a statement of code multiple times in succession. By using variables that can be assigned new values, each repetition of the statement may be different. The following loop executes exactly 10 times: **Türkçe:** Sırada loop'lar var. Loop, bir code statement'ını art arda birden çok kez çalıştırabilen tekrarlı control structure'dır. Yeni değer atanabilen variable'lar sayesinde statement'ın her tekrarı farklı olabilir. Aşağıdaki loop tam 10 kez çalışır:

```
int counter = 0;
while (counter < 10) {
    double price = counter * 10;
    System.out.println(price);
    counter++;
}
```

English: If you don't follow this code, don't panic—we cover it shortly. In this section, we're going to discuss the while loop and its two forms. In the next section, we move on to for loops, which have their roots in while loops. **Türkçe:** Bu kodu henüz takip edemiyorsanız endişelenmeyin; birazdan ele alacağız. Bu bölümde while loop'u ve iki biçimini inceleyeceğiz. Sonraki bölümde kökeni while loop'larına dayanan for loop'larına geçeceğiz.

The while Statement

Türkçe başlık: `while` Statement'i

English: The simplest repetitive control structure in Java is the while statement, described in Figure 3.5. Like all repetition control structures, it has a termination condition, implemented as a boolean expression, that will continue as long as the expression evaluates to true. **Türkçe:** Java'daki en basit tekrarlı control structure, Şekil 3.5'te açıklanan `while` statement'ıdır. Bütün tekrarlı control structure'lar gibi bunun da boolean expression olarak uygulanan bir termination condition'ı vardır; expression `true` ürettiği sürece loop devam eder.

FIGURE 3.5 The structure of a while statement

Türkçe başlık: ŞEKİL 3.5 Bir `while` statement'ının yapısı

English: while keyword Parentheses (required) **Türkçe:** `while` keyword'ü; parantez zorunludur.

```
while (booleanExpression) {  
    // Body
```

English: Curly braces required for block of multiple statements, optional for single statement **Türkçe:** Birden fazla ifadenin bloğu için küme parantezleri gereklidir, tek ifade için isteğe bağlıdır

```
}
```

English: As shown in Figure 3.5, a while loop is similar to an if statement in that it is composed of a boolean expression and a statement, or a block of statements. During execution, the boolean expression is evaluated before each iteration of the loop and exits if the evaluation returns false. **Türkçe:** Şekil 3.5'te gösterildiği gibi `while` loop'u, boolean expression ile tek bir statement'tan veya statement block'undan oluşması bakımından `if` statement'ına benzer. Runtime'da boolean expression loop'un her iteration'ından önce değerlendirilir; sonuç `false` ise loop sona erer.

English: Let's see how a loop can be used to model a mouse eating a meal: **Türkçe:** Yemek yiyen bir fareyi modellemek için bir loop'un nasıl kullanılabileceğini görelim:

```
int roomInBelly = 5;  
public void eatCheese(int bitesOfCheese) {  
    while (bitesOfCheese > 0 && roomInBelly > 0) {  
        bitesOfCheese--;  
        roomInBelly--;  
    }  
    System.out.println(bitesOfCheese+" pieces of cheese left");  
}
```

English: This method takes an amount of food—in this case, cheese—and continues until the mouse has no room in its belly or there is no food left to eat. With each iteration of the loop, the mouse “eats” one bite of food and loses one spot in its belly. By using a compound boolean statement, you ensure that the while loop can end for either of the conditions. **Türkçe:** Bu method belirli miktarda yiyecek—bu örnekte peynir—alır ve farenin karnında yer kalmayana ya da yiyecek bitene kadar sürer. Loop'un her iteration'ında fare bir lokma yer ve karnındaki bir birimlik yeri kaybeder. Compound boolean expression kullanarak `while` loop'unun iki condition'dan biri nedeniyle sona erebilmesini sağlarsınız.

English: One thing to remember is that a while loop may terminate after its first evaluation of the boolean expression. For example, how many times is `Not full!` printed in the following example? **Türkçe:** `while` loop'unun boolean expression ilk kez değerlendirildikten hemen sonra sona erebileceğini unutmayın. Örneğin aşağıdaki kod `Not full!` metnini kaç kez yazdırır?

```
int full = 5;
while(full < 5) {
    System.out.println("Not full!");
    full++;
}
```

English: The answer? Zero! On the first iteration of the loop, the condition is reached, and the loop exits. This is why while loops are often used in places where you expect zero or more executions of the loop. Simply put, the body of the loop may not execute at all or may execute many times. **Türkçe:** Cevap sıfırdır. İlk iteration'da condition değerlendirilir ve loop sona erer. Bu nedenle `while` loop'ları, gövdenin sıfır veya daha fazla kez çalışmasının beklendiği yerlerde kullanılır. Kısacası loop body hiç çalışmayabilir ya da birçok kez çalışabilir.

The do/while Statement

Türkçe başlık: `do/while` Statement'ı

English: The second form a while loop can take is called a do/while loop, which, like a while loop, is a repetition control structure with a termination condition and statement, or a block of statements, as shown in Figure 3.6. **Türkçe:** `while` loop'unun ikinci biçimi `do/while` loop'udur. Şekil 3.6'da gösterildiği gibi bu da bir termination condition ile tek bir statement veya statement block içeren bir repetition control structure'dır.

FIGURE 3.6 The structure of a do/while statement

Türkçe başlık: ŞEKİL 3.6 Bir `do/while` statement'ının yapısı

English: `do` keyword **Türkçe:** `do` keyword'ü

```
do {
```

English: Curly braces required for block of multiple statements, optional for single statement **Türkçe:** Birden fazla ifadenin bloğu için küme parantezleri gereklidir, tek ifade için isteğe bağlıdır

```
// Body
} while (booleanExpression);
```

English: Semicolon (required) while keyword Parentheses (required) Unlike a while loop, though, a do/while loop guarantees that the statement or block will be executed at least once. For example, what is the output of the following statements? **Türkçe:** Noktalı virgül zorunludur · `while` keyword'ü · Parantezler zorunludur. `while` loop'undan farklı olarak `do/while` loop, statement'ın veya block'un en az bir kez çalıştırılacağını garanti eder. Örneğin aşağıdaki statement'ların output'u nedir?

```
int lizard = 0;
do {
    lizard++;
} while(false);
System.out.println(lizard); // 1
```

English: Java will execute the statement block first and then check the loop condition. Even though the loop exits right away, the statement block is still executed once, and the program prints 1. **Türkçe:** Java önce statement block'unu çalıştırır, ardından loop condition'ını kontrol eder. Loop hemen sona erse bile block bir kez çalışmıştır; bu nedenle program 1 yazdırır.

Infinite Loops

Türkçe başlık: Infinite Loop'lar

English: The single most important thing you should be aware of when you are using any repetition control structures is to make sure they always terminate! Failure to terminate a loop can lead to numerous problems in practice, including overflow exceptions, memory leaks, slow performance, and even bad data. Let's take a look at an example: **Türkçe:** Tekrarlı bir control structure kullanırken dikkat edilmesi gereken en önemli nokta, yapının mutlaka sona ermesidir. Bir loop'un sona ermemesi overflow exception'ları, memory leak'ler, düşük performans ve hatalı veriler dahil birçok soruna yol açabilir. Bir örneğe bakalım:

```
int pen = 2;
int pigs = 5;
while(pen < 10)
    pigs++;
```

English: You may notice one glaring problem with this statement: it will never end. The variable `pen` is never modified, so the expression `(pen < 10)` will always evaluate to true. The result is that the loop will never end, creating what is commonly referred to as an infinite loop. An infinite loop is a loop whose termination condition is never reached during runtime. **Türkçe:** Bu statement'ta açık bir sorun vardır: Hiç sona ermez. `pen` variable'ı değiştirilmediği için `(pen < 10)` expression'ı her zaman `true` olur. Sonuçta loop hiç bitmez ve infinite loop oluşur. Infinite loop, termination condition'ına runtime'da hiçbir zaman ulaşamayan loop'tur.

English: Anytime you write a loop, you should examine it to determine whether the termination condition is always eventually met under some condition. For example, a loop in which no variables are changing between two executions suggests that the termination condition may not be met. The loop variables should always be moving in a particular direction. **Türkçe:** Bir loop yazdığınızda termination condition'ın en az bir koşul altında sağlanıp sağlanamayacağını inceleyin. İki execution arasında hiçbir variable'ın değişmediği bir loop, termination condition'a ulaşamayabileceğini gösterir. Loop variable'ları belirli bir yönde ilerlemelidir.

English: In other words, make sure the loop condition, or the variables the condition is dependent on, are changing between executions. Then, ensure that the termination condition will be eventually reached in all circumstances. As you learn in the last section of this chapter, a loop may also exit under other conditions, such as a break statement. **Türkçe:** Başka bir deyişle loop condition'ın veya bu condition'ın bağlı olduğu variable'ların iteration'lar arasında değiştiğinden emin olun. Ardından bütün durumlarda termination condition'a eninde sonunda ulaşılabilirdiğini doğrulayın. Bölümün sonunda göreceğiniz gibi loop, break statement'ı gibi başka yollarla da sonlandırılabilir.

Constructing for Loops

Türkçe başlık: for Loop'larını Oluşturma

English: Even though while and do/while statements are quite powerful, some tasks are so common in writing software that special types of loops were created—for example, iterating over a statement exactly 10 times or iterating over a list of names. You could easily accomplish these tasks with various while loops that you've seen so far, but they usually require a lot of boilerplate code. Wouldn't it be great if there was a looping structure that could do the same thing in a single line of code? **Türkçe:** while ve do/while statement'ları güçlüdür; ancak bir statement'ı tam 10 kez çalıştırmak veya ad listesindeki element'lar üzerinde iteration yapmak gibi bazı görevler çok yaygındır. Bunları çeşitli while loop'larıyla yapabilirsiniz; fakat çoğunlukla fazla boilerplate code gerekir. Aynı işi tek satırda yapan bir loop yapısı daha kullanışlı olmaz mıydı?

English: With that, we present the most convenient repetition control structure, for loops. There are two types of for loops, although both use the same for keyword. The first is referred to as the basic for loop, and the second is often called the enhanced for loop. For clarity, we refer to them as the for loop and the for-each loop, respectively, throughout the book. **Türkçe:** Şimdi en kullanışlı tekrarlı control structure olan for loop'larını tanıtıyoruz. İki for loop türü vardır ve ikisi de for keyword'ünü kullanır. İlki basic for loop, ikincisi çoğunlukla enhanced for loop olarak adlandırılır. Kitap boyunca açıklık için bunlara sırasıyla for loop ve for-each loop diyeceğiz.

The for Loop

Türkçe başlık: for Loop'u

English: A basic for loop has the same conditional boolean expression and statement, or block of statements, as the while loops, as well as two new sections: an initialization block and an update statement. Figure 3.7 shows how these components are laid out. **Türkçe:** Temel `for` loop'u, `while` loop'undaki conditional boolean expression ile statement veya statement block'una ek olarak iki yeni bölüme sahiptir: initialization block ve update statement. Şekil 3.7 bu bileşenlerin yerleşimini gösterir.

English: Although Figure 3.7 might seem a little confusing and almost arbitrary at first, the organization of the components and flow allow us to create extremely powerful statements in a single line that otherwise would take multiple lines with a while loop. Each of the three sections is separated by a semicolon. In addition, the initialization and update sections may contain multiple statements, separated by commas.

Türkçe: Şekil 3.7 ilk bakışta biraz karmaşık ve gelişigüzel görünse de bileşenlerin ve control flow'un bu düzeni, `while` loop'uyla birden çok satır gerektirecek güçlü yapıları tek satırda yazmamızı sağlar. Üç bölüm noktalı virgüllerle ayrılır. Initialization ve update bölümleri ayrıca virgülle ayrılmış birden çok expression içerebilir.

English: Variables declared in the initialization block of a for loop have limited scope and are accessible only within the for loop. Be wary of any exam questions in which a variable is declared within the initialization block of a for loop and then read outside the loop. For example, this code does not compile because the loop variable `i` is referenced outside the loop: **Türkçe:** Bir `for` loop'un initialization block'unda bildirilen variable'ların scope'u sınırlıdır; bunlara yalnızca `for` loop içinde erişilebilir. Bir variable'ın initialization block'ta bildirilip loop dışında okunduğu sınav sorularına dikkat edin. Örneğin bu kod, loop variable `i`ye loop dışında reference verildiği için derlenmez:

FIGURE 3.7 The structure of a basic for loop

Türkçe başlık: ŞEKİL 3.7 Temel `for` loop'unun yapısı

English: for keyword Parentheses (required) Semicolons (required) **Türkçe:** `for` keyword'ü · Parantezler zorunludur · Noktalı virgüller zorunludur

```
for (initialization; booleanExpression; updateStatement) {
    // Body
}
```

English: Curly braces required for block of multiple statements, optional for single statement 1 Initialization statement executes 2 If booleanExpression is true, continue; else exit loop 3 Body executes 4 Execute updateStatement **Türkçe:** Birden çok statement içeren block için küme parantezleri zorunlu, tek statement için isteğe bağlıdır. 1 Initialization statement çalışır. 2 `booleanExpression true` ise devam edilir; değilse loop'tan çıkılır. 3 Body çalışır. 4 `updateStatement` çalışır.

5 Return to Step 2

Türkçe başlık: 5. 2. adıma dön

```
for(int i=0; i < 10; i++)  
System.out.println("Value is: "+i);  
System.out.println(i); // DOES NOT COMPILE
```

English: Alternatively, variables declared before the for loop and assigned a value in the initialization block may be used outside the for loop because their scope precedes the creation of the for loop. **Türkçe:** Buna karşılık `for` loop'tan önce bildirilen ve initialization block'ta değer atanan variable'lar, scope'ları loop oluşturulmadan önce başladığı için `for` loop dışında kullanılabilir.

```
int i;  
for(i=0; i < 10; i++)  
System.out.println("Value is: "+i);  
System.out.println(i);
```

English: Let's take a look at an example that prints the first five numbers, starting with zero: **Türkçe:** Sıfırdan başlayarak ilk beş rakamı yazdıran bir örneğe bakalım:

```
for(int i = 0; i < 5; i++) {  
System.out.print(i + " ");  
}
```

English: The local variable `i` is initialized first to 0. The variable `i` is only in scope for the duration of the loop and is not available outside the loop once the loop has completed. Like a while loop, the boolean condition is evaluated on every iteration of the loop before the loop executes. Since it returns true, the loop executes and outputs 0 followed by a space. Next, the loop executes the update section, which in this case increases the value of `i` to 1. The loop then evaluates the boolean expression a second time, and the process repeats multiple times, printing the following: **Türkçe:** Local variable `i` önce 0 ile initialize edilir. `i` yalnızca loop boyunca scope içindedir; loop tamamlandıktan sonra loop dışında kullanılamaz. `while` loop'unda olduğu gibi boolean condition, body çalışmadan önce her iteration'da değerlendirilir. Sonuç `true` olduğundan loop çalışır ve 0 ile ardından bir boşluk yazdırır. Sonra update bölümü çalışır ve bu örnekte `i` değeri 1 olur. Ardından boolean expression ikinci kez değerlendirilir ve süreç tekrarlanarak şu output üretilir:

English: 0 1 2 3 4 **Türkçe:** 0 1 2 3 4

English: On the fifth iteration of the loop, the value of `i` reaches 4 and is incremented by 1 to reach 5. On the sixth iteration of the loop, the boolean expression is evaluated, and since $(5 < 5)$ returns false, the loop terminates without executing the statement loop body. **Türkçe:** Loop'un beşinci iteration'ında `i` değeri 4 olur ve 1 artırılarak 5'e ulaşır. Altıncı iteration'da boolean expression değerlendirilir; $(5 < 5)$ sonucu `false` olduğundan loop body çalıştırılmadan loop sona erer.

English: Why `i` in for Loops? **Türkçe:** `for` Loop'larında Neden `i` Kullanılır?

English: You may notice it is common practice to name a for loop variable `i`. Long before Java existed, programmers started using `i` as short for increment variable, and the practice exists today, even though many of those programming languages no longer do! For double or triple loops, where `i` is already used, the next letters in the alphabet, `j` and `k`, are often used. **Türkçe:** Bir `for` loop variable'ına `i` adını vermenin yaygın olduğunu fark edebilirsiniz. Java ortaya çıkmadan çok önce programcılar `i` harfini increment variable'ın kısaltması olarak kullanmaya başladı; bu gelenek bugün de sürer. `i`'nin zaten kullanıldığı çift veya üçlü nested loop'larda çoğunlukla alfabedeki sonraki harfler olan `j` ve `k` kullanılır.

Printing Elements in Reverse

Türkçe başlık: Elementleri Ters Sırada Yazdırma

English: Let's say you wanted to print the same first five numbers from zero as we did in the previous section, but this time in reverse order. The goal then is to print 4 3 2 1 0. **Türkçe:** Diyelim ki önceki bölümde yaptığımız gibi sıfırdan başlayan ilk beş sayıyı bu sefer ters sırada yazdırmak istediniz. O zaman amaç 4 3 2 1 0'u basmaktır.

English: How would you do that? An initial implementation might look like the following: **Türkçe:** Bunu nasıl yapabiliriz? İlk implementasyon şöyle görünebilir:

```
for (var counter = 5; counter > 0; counter--) {  
    System.out.print(counter + " ");  
}
```

English: While this snippet does output five distinct values, and it resembles our first for loop example, it does not output the same five values. Instead, this is the output: **Türkçe:** Bu code snippet beş farklı değer üretir ve ilk `for` loop örneğimize benzer; ancak aynı beş değeri yazdırmaz. Ürettiği output şudur:

English: 5 4 3 2 1 Wait, that's not what we wanted! We wanted 4 3 2 1 0. It starts with 5, because that is the first value assigned to it. Let's fix that by starting with 4 instead: **Türkçe:** 5 4 3 2 1 Durun, istediğimiz bu değildi! Biz 4 3 2 1 0'ı istedik. 5 ile başlıyor çünkü ona atanan ilk değer bu. Bunun yerine 4 ile başlayarak bunu düzeltelim:

```
for (var counter = 4; counter > 0; counter--) {  
    System.out.print(counter + " ");  
}
```

English: What does this print now? It prints the following: **Türkçe:** Bu şimdi ne yazdırıyor? Aşağıdakileri yazdırır:

English: 4 3 2 1 So close! The problem is that it ends with 1, not 0, because we told it to exit as soon as the value was not strictly greater than 0. If we want to print the same 0 through 4 as our first example, we need to update the termination condition, like this: **Türkçe:** 4 3 2 1 Çok yakın! Sorun şu ki 0 ile değil 1 ile bitiyor çünkü değer 0'dan büyük olmadığı anda çıkmasını söyledik. İlk örneğimizdeki gibi 0'dan 4'e kadar olan değerlerin aynısını yazdırmak istiyorsak, sonlandırma koşulunu şu şekilde güncellememiz gerekir:

```
for (var counter = 4; counter >= 0; counter--) {  
    System.out.print(counter + " ");  
}
```

English: Finally! We have code that now prints 4 3 2 1 0 and matches the reverse of our for loop example in the previous section. We could have instead used `counter > -1` as the loop termination condition in this example, although `counter >= 0` tends to be more readable. **Türkçe:** Sonunda! Artık 4 3 2 1 0 değerini yazdıran ve önceki bölümdeki for loop örneğimizin tersiyle eşleşen bir kodumuz var. Bunun yerine bu örnekte loop sonlandırma koşulu olarak `counter > -1`'i kullanabilirdik, ancak `counter >= 0` daha okunabilir olma eğilimindedir.

English: For the exam, you are going to have to know how to read forward and backward for loops. When you see a for loop on the exam, pay close attention to the loop variable and operations if the decrement operator, `--`, is used. While incrementing from 0 in a for loop is often straightforward, decrementing tends to be less intuitive. In fact, if you do see a for loop with a decrement operator on the exam, you should assume they are trying to test your knowledge of loop operations. **Türkçe:** Sınav için ileri ve geri ilerleyen `for` loop'larını okuyabilmelisiniz. Decrement operator (`--`) kullanılıyorsa loop variable'ına ve işlemlere özellikle dikkat edin. Sıfırdan başlayıp artırmak çoğunlukla kolaydır; decrement işlemleri ise daha az sezgiseldir. Sınavda decrement operator içeren bir `for` loop'u görürseniz loop işlemlerinin sınındığını varsayabilirsiniz.

Working with for Loops

Türkçe başlık: `for` Loop'larıyla Çalışma

English: Although most for loops you are likely to encounter in your professional development experience will be well defined and similar to the previous examples, there are a number of variations and edge cases you could see on the exam. You should familiarize yourself with the following five examples; variations of these are likely to be seen on the exam. **Türkçe:** Profesyonel geliştirmede karşılaştığınız `for` loop'larının çoğu iyi tanımlanmış ve önceki örneklere benzerdir; fakat sınavda çok sayıda variation ve edge case görebilirsiniz. Aşağıdaki beş örneğe aşina olun; sınavda bunların variation'larıyla karşılaşmanız olasıdır.

English: Let's tackle some examples for illustrative purposes: **Türkçe:** Açıklama amacıyla bazı örnekleri ele alalım:

1. Creating an Infinite Loop

Türkçe başlık: 1. Infinite Loop Oluşturma

```
for( ; ; )  
    System.out.println("Hello World");
```

English: Although this for loop may look like it does not compile, it will in fact compile and run without issue. It is actually an infinite loop that will print the same statement repeatedly. **Türkçe:** Bu `for` loop ilk bakışta derlenmeyecekmiş gibi görünse de aslında derlenir ve sorunsuz çalışır. Aynı expression'ı tekrar tekrar yazdıran infinite loop'tur.

English: This example reinforces the fact that the components of the for loop are each optional. **Türkçe:** Bu örnek, for loop bileşenlerinin her birinin isteğe bağlı olduğu gerçeğini güçlendirir.

English: Note that the semicolons separating the three sections are required, as `for( )` without any semicolons will not compile. **Türkçe:** Üç bölümü ayıran noktalı virgüllerin zorunlu olduğunu unutmayın; hiçbir noktalı virgül içermeyen `for( )` derlenmez.

2. Adding Multiple Terms to the for Statement

Türkçe başlık: 2. `for` Statement'ına Birden Çok Terim Ekleme

```
int x = 0;
for(long y = 0, z = 4; x < 5 && y < 10; x++, y++) {
    System.out.print(y + " ");
    System.out.print(x + " ");
}
```

English: This code demonstrates three variations of the for loop you may not have seen. **Türkçe:** Bu kod, for loop'unun görmemiş olabileceğiniz üç varyasyonunu gösterir.

English: First, you can declare a variable, such as `x` in this example, before the loop begins and use it after it completes. Second, your initialization block, boolean expression, and update statements can include extra variables that may or may not reference each other. **Türkçe:** İlk olarak bu örnekteki `x` gibi bir variable'ı loop başlamadan önce bildirip loop tamamlandıktan sonra kullanabilirsiniz. İkinci olarak initialization block, boolean expression ve update statement'lar birbirine reference verebilen veya vermeyen ek variable'lar içerebilir.

English: other. For example, `z` is defined in the initialization block and is never used. Finally, the update statement can modify multiple variables. This code will print the following when executed: **Türkçe:** birbirine reference verebilir veya vermeyebilir. Örneğin `z`, initialization block'ta bildirilir ve hiç kullanılmaz. Son olarak update statement birden fazla variable'ı değiştirebilir. Kod çalıştırıldığında şu output'u üretir:

English: 0 1 2 3 4 5 **Türkçe:** 0 1 2 3 4 5

Redeclaring a Variable in the Initialization Block

Türkçe başlık: Başlatma Bloğunda Bir Değişkeni Yeniden Bildirmek

```
int x = 0;
for(int x = 4; x < 5; x++) // DOES NOT COMPILE
    System.out.print(x + " ");
```

English: This example looks similar to the previous one, but it does not compile because of the initialization block. The difference is that `x` is repeated in the initialization block after already being declared before the loop, resulting in the compiler stopping because of a duplicate variable declaration. We can fix this loop by removing the declaration of `x` from the for loop as follows: **Türkçe:** Bu örnek öncekine benzer; ancak initialization block nedeniyle derlenmez. Buradaki fark, `x` loop'tan önce bildirildiği hâlde initialization block'ta yeniden bildirilmesidir. Duplicate variable declaration compiler error üretir. Initialization'daki `x` bildirimini kaldırarak loop'u düzeltebiliriz:

```
int x = 0;
for(x = 0; x < 5; x++)
    System.out.print(x + " ");
```

English: Note that this variation will now compile because the initialization block simply assigns a value to `x` and does not declare it. **Türkçe:** Başlatma bloğunun `x`'e basitçe bir değer ataması ve bunu bildirmemesi nedeniyle bu varyasyonun şimdi derleneceğini unutmayın.

Using Incompatible Data Types in the Initialization Block

Türkçe başlık: Başlatma Bloğunda Uyumsuz Veri Türlerini Kullanma

```
int x = 0;
for(long y = 0, int z = 4; x < 5; x++) // DOES NOT COMPILE
    System.out.print(y + " ");
```

English: Like the third example, this code will not compile, although this time for a different reason. The variables in the initialization block must all be of the same type. In the multiple-terms example, `y` and `z` were both long, so the code compiled without issue; but in this example, they have different types, so the code will not compile. **Türkçe:** Üçüncü örnekteki gibi bu kod da derlenmez; ancak nedeni farklıdır. Initialization block'taki variable'ların tümü aynı type'ta olmalıdır. Multiple-terms örneğinde `y` ile `z` aynı long type'ında olduğundan kod sorunsuz derlenmişti; ancak bu örnekte type'ları farklı olduğundan kod derlenmez.

Using Loop Variables Outside the Loop

Türkçe başlık: Loop Dışındaki Variable'ları Kullanma

```
for(long y = 0, x = 4; x < 5 && y < 10; x++, y++)
    System.out.print(y + " ");
System.out.print(x); // DOES NOT COMPILE
```

English: We covered this already at the start of this section, but it is so important for passing the exam that we discuss it again here. If you notice, `x` is defined in the initialization block of the loop and then used after the loop terminates. Since `x` was only scoped for the loop, using it outside the loop will cause a compiler error. **Türkçe:** Bu konuyu bölümün başında ele aldık; ancak sınav açısından çok önemli olduğundan yeniden vurguluyoruz. `x`, loop'un initialization block'unda bildirilip loop sona erdikten sonra kullanılıyor. `x` yalnızca loop scope'unda olduğundan loop dışında kullanılması compilation error'a yol açar.

Modifying Loop Variables

Türkçe başlık: Loop Variable'larını Değiştirme

English: As a general rule, it is considered a poor coding practice to modify loop variables due to the unpredictability of the result, such as in the following examples: **Türkçe:** Genel bir kural olarak loop variable'larını değiştirmek, aşağıdaki örneklerde görüldüğü gibi sonucu öngörmeyi zorlaştırdığından kötü bir coding practice kabul edilir:

```
for(int i=0; i<10; i++)  
i = 0;  
for(int j=1; j<10; j++)  
j++;
```

English: It also tends to make code difficult for other people to follow. **Türkçe:** Ayrıca diğer insanların kodu takip etmesini zorlaştırma eğilimindedir.

The for-each Loop

Türkçe başlık: `for-each` Loop'u

English: The for-each loop is a specialized structure designed to iterate over arrays and various Collections Framework classes, as presented in Figure 3.8. **Türkçe:** `for-each` loop, Şekil 3.8'de gösterildiği gibi array'ler ve çeşitli Collections Framework class'ları üzerinde iteration yapmak için tasarlanmış özel bir yapıdır.

FIGURE 3.8 The structure of an enhanced for-each loop

Türkçe başlık: ŞEKİL 3.8 Enhanced `for-each` loop'unun yapısı

English: for keyword Parentheses (required) Colon (required) **Türkçe:** `for` keyword'ü · Parantezler zorunludur · İki nokta (:) zorunludur

```
for (datatype instance: collection) {
```

English: Iterable collection of objects **Türkçe:** Object'lerden oluşan `Iterable` collection

```
// Body
```

English: datatype of collection member **Türkçe:** Collection element'inin data type'ı

```
}
```

English: Curly braces required for block of multiple statements, optional for single statement The for-each loop declaration is composed of an initialization section and an object to be iterated over. The right side of the for-each loop must be one of the following: **Türkçe:** Birden çok statement içeren block için küme parantezleri zorunlu, tek statement için isteğe bağlıdır. `for-each` loop bildirimi bir initialization bölümü ile üzerinde iteration yapılacak object'ten oluşur. `for-each` loop'un sağ tarafı şunlardan biri olmalıdır:

English: ■ ■ A built-in Java array **Türkçe:** ■ ■ Yerleşik bir Java array'i

English: ■ ■ An object whose type implements `java.lang.Iterable` We cover what implements means in Chapter 7, but for now you just need to know that the right side must be an array or collection of items, such as a List or a Set. For the exam, you should know that this does not include all of the Collections Framework classes **Türkçe:** ■ ■ Type'i `java.lang.Iterable` implement eden bir object. `implements` kavramını Bölüm 7'de ele alacağız; şimdilik sağ tarafın bir array veya `List` ya da `Set` gibi bir element collection'ı olması gerektiğini bilmeniz yeterlidir. Sınav için bunun bütün Collections Framework class'larını

English: or interfaces, but only those that implement or extend that Collection interface. For example, Map is not supported in a for-each loop, although Map does include methods that return Collection instances. **Türkçe:** ve interface'lerini kapsamadığını; yalnızca `Collection` interface'ini implement veya extend edenleri kapsadığını bilmelisiniz. Örneğin `Map`, `Collection` instance'ları döndüren method'lar içerse de `for-each` loop'unda doğrudan desteklenmez.

English: The left side of the for-each loop must include a declaration for an instance of a variable whose type is compatible with the type of the array or collection on the right side of the statement. On each iteration of the loop, the named variable on the left side of the statement is assigned a new value from the array or collection on the right side of the statement. **Türkçe:** `for-each` loop'un sol tarafı, type'i statement'ın sağındaki array veya collection element type'ıyla uyumlu bir variable bildirimi içermelidir. Her iteration'da sağ taraftaki array veya collection'dan alınan yeni değer, sol tarafta adı verilen variable'a atanır.

English: Compare these two methods that both print the values of an array, one using a traditional for loop and the other using a for-each loop: **Türkçe:** Her ikisi de bir array'in değerlerini basan bu iki method'u karşılaştırın; biri geleneksel loop, diğeri ise `for-each` loop kullanıyor:

```
public void printNames(String[] names) {
    for(int counter=0; counter<names.length; counter++)
        System.out.println(names[counter]);
}
public void printNames(String[] names) {
    for(var name : names)
        System.out.println(name);
}
```

English: The for-each loop is a lot shorter, isn't it? We no longer have a counter loop variable that we need to create, increment, and monitor. Like using a for loop in place of a while loop, for-each loops are meant to reduce boilerplate code, making code easier to read/write, and freeing you to focus on the parts of your code that really matter. **Türkçe:** `for-each` loop çok daha kısa, değil mi? Artık oluşturmamız, artırmamız ve izlememiz gereken bir sayaç loop değişkenimiz yok. Bir süre loop yerine for loop kullanmak gibi, for-each loop'ları da ortak kodu azaltmak, kodun okunmasını/yazılmasını kolaylaştırmak ve kodunuzun gerçekten önemli bölümlerine odaklanmanızı sağlamak için tasarlanmıştır.

English: We can also use a for-each loop on a List, since it implements Iterable. **Türkçe:** `List`, `Iterable` implement ettiği için bir `List` üzerinde de `for-each` loop kullanabiliriz.

```
public void printNames(List<String> names) {
    for(var name : names)
        System.out.println(name);
}
```

English: We cover generics in detail in Chapter 9, "Collections and Generics." For this chapter, you just need to know that on each iteration, a for-each loop assigns a variable with the same type as the generic argument. In this case, name is of type String. **Türkçe:** Generics konusunu Bölüm 9, "Collections and Generics" içinde ayrıntılı ele alıyoruz. Bu bölüm için her iteration'da `for-each` loop variable'ına generic argument ile aynı type'ta bir değer atandığını bilmeniz yeterlidir. Bu örnekte `name` variable'ının type'ı `String`dir.

English: So far, so good. What about the following examples? **Türkçe:** Şu ana kadar çok iyi. Aşağıdaki örneklere ne dersiniz?

```
String birds = "Jay";
for(String bird : birds) // DOES NOT COMPILE
    System.out.print(bird + " ");
String[] sloths = new String[3];
for(int sloth : sloths) System.out.print(sloth + " ");
// DOES NOT COMPILE
```

English: The first for-each loop does not compile because String cannot be used on the right side of the statement. While a String may represent a list of characters, it has to actually be an array or implement Iterable. The second example does not compile because the loop type on the left side of the statement is int and doesn't match the expected type of String. **Türkçe:** İlk `for-each` loop derlenmez; çünkü statement'ın sağ tarafında `String` kullanılamaz. `String` bir character listesi gibi görünse de sağ taraf gerçekte bir array olmalı veya `Iterable` implement etmelidir. İkinci örnek de statement'ın solundaki loop variable type'ı `int` olup beklenen `String` type'ıyla eşleşmediği için derlenmez.

Controlling Flow with Branching

Türkçe başlık: Dallanma ile Akışı Kontrol Etme

English: The final types of control flow structures we cover in this chapter are branching statements. **Türkçe:** Bu bölümde ele aldığımız son control-flow structure türü branching statement'lardır.

English: Up to now, we have been dealing with single loops that ended only when their boolean expression evaluated to false. We now show you other ways loops could end, or branch, and you see that the path taken during runtime may not be as straightforward as in the previous examples. **Türkçe:** Şimdiye kadar yalnızca boolean expression'ı `false` olduğunda sona eren tekil loop'ları ele aldık. Şimdi loop'ların sona ermesini veya farklı bir execution path'e dallanmasını sağlayan diğer yolları göstereceğiz; runtime'da izlenen yol önceki örneklerdeki kadar basit olmayabilir.

Nested Loops

Türkçe başlık: Nested Loop'lar

English: Before we move into branching statements, we need to introduce the concept of nested loops. A nested loop is a loop that contains another loop, including while, do/while, for, and for-each loops. For example, consider the following code that iterates over a two-dimensional array, which is an array that contains other arrays as its members. We cover multidimensional arrays in detail in Chapter 4, “Core APIs,” but for now, assume the following is how you would declare a two-dimensional array: **Türkçe:** Branching statement'lara geçmeden önce nested loop kavramını tanıtalım. Nested loop; `while`, `do/while`, `for` veya `for-each` dahil başka bir loop içeren loop'tur. Örneğin, element'ları başka array'ler olan iki boyutlu bir array üzerinde iteration yapan aşağıdaki kodu düşünün. Multidimensional array'leri Bölüm 4, “Core APIs” içinde ayrıntılı ele alacağız; şimdilik iki boyutlu bir array'in şöyle bildirildiğini kabul edin:

```
int[][] myComplexArray = {{5,2,1,3},{3,9,8,9},{5,7,12,7}};
for(int[] mySimpleArray : myComplexArray) {
    for(int i=0; i<mySimpleArray.length; i++) {
        System.out.print(mySimpleArray[i]+"\\t");
    }
    System.out.println();
}
```

English: Notice that we intentionally mix a for loop and a for-each loop in this example. The outer loop will execute a total of three times. Each time the outer loop executes, the inner loop is executed four times. When we execute this code, we see the following output: **Türkçe:** Bu örnekte bir `for` loop ile bir `for-each` loop'u bilerek birlikte kullandığımıza dikkat edin. Outer loop toplam üç kez, her outer-loop iteration'ında inner loop dört kez çalışır. Kod çalıştırıldığında şu output üretilir:

English: 5 2 1 3 3 9 8 9 5 7 12 7 Nested loops can include while and do/while, as shown in this example. See whether you can determine what this code will output: **Türkçe:** 5 2 1 3 3 9 8 9 5 7 12 7 Nested loop'lar, bu örnekteki gibi `while` ve `do/while` loop'larını da içerebilir. Kodun hangi output'u üreteceğini bulmaya çalışın:

```
int hungryHippopotamus = 8;
while(hungryHippopotamus>0) {
    do {
        hungryHippopotamus -= 2;
    } while (hungryHippopotamus>5);
    hungryHippopotamus--;
    System.out.print(hungryHippopotamus+"\\n");
}
```

English: The first time this loop executes, the inner loop repeats until the value of `hungryHippopotamus` is 4. The value will then be decremented to 3, and that will be the output at the end of the first iteration of the outer loop. **Türkçe:** Outer loop ilk kez çalıştığında inner loop, `hungryHippopotamus` değeri 4 olana kadar yinelenir. Ardından değer 3'e düşürülür; outer loop'un ilk iteration'ı sonunda yazdırılan değer de budur.

English: On the second iteration of the outer loop, the inner `do/while` will be executed once, even though `hungryHippopotamus` is already not greater than 5. As you may recall, `do/while` statements always execute the body at least once. This will reduce the value to 1, which will be further lowered by the decrement operator in the outer loop to 0. Once the value reaches 0, the outer loop will terminate. The result is that the code will output the following: **Türkçe:** Outer loop'un ikinci iteration'ında `hungryHippopotamus` zaten 5'ten büyük olmasa bile inner `do/while` bir kez çalışır. `do/while` statement body'yi her zaman en az bir kez çalıştırır. Böylece değer önce 1'e, ardından outer loop'taki decrement operator ile 0'a düşer. Değer 0 olduğunda outer loop sona erer ve kod şu output'u üretir:

English: 3, 0, The examples in the rest of this section include many nested loops. You will also encounter nested loops on the exam, so the more practice you have with them, the more prepared you will be. **Türkçe:** 3, 0, Bu bölümün geri kalanındaki örnekler çok sayıda nested loop içerir. Sınavda da nested loop'larla karşılaşacağınız için ne kadar çok pratik yaparsanız o kadar hazırlıklı olursunuz.

Adding Optional Labels

Türkçe başlık: İsteğe Bağlı Etiketler Ekleme

English: One thing we intentionally skipped when we presented if statements, switch statements, and loops is that they can all have optional labels. A label is an optional pointer to the head of a statement that allows the application flow to jump to it or break from it. It is a single identifier that is followed by a colon (:). For example, we can add optional labels to one of the previous examples: **Türkçe:** `if` ve `switch` statement'ları ile loop'ları tanıtırken hepsinin isteğe bağlı label alabileceğini özellikle sonraya bıraktık. Label, application flow'un statement'ın başına atlamasını veya ondan çıkmasını sağlayan isteğe bağlı bir işaretçidir; ardından iki nokta (:) gelen tek bir identifier'dır. Önceki örneklerden birine isteğe bağlı label'lar ekleyebiliriz:

```
int[][] myComplexArray = {{5,2,1,3},{3,9,8,9},{5,7,12,7}};
```

English: OUTER_LOOP: for(int[] mySimpleArray : myComplexArray) { **Türkçe:** OUTER_LOOP: for(int[] mySimpleArray : myComplexArray) {

```
INNER_LOOP: for(int i=0; i<mySimpleArray.length; i++) {
    System.out.print(mySimpleArray[i]+"\\t");
}
System.out.println();
}
```

English: Labels follow the same rules for formatting as identifiers. For readability, they are commonly expressed using uppercase letters in snake_case with underscores between words. **Türkçe:** Label'lar identifier'larla aynı biçimlendirme kurallarına uyar. Okunabilirlik için çoğunlukla büyük harfli SNAKE_CASE biçiminde, sözcükler arasında underscore kullanılarak yazılırlar.

English: When dealing with only one loop, labels do not add any value, but as you learn in the next section, they are extremely useful in nested structures. **Türkçe:** Yalnızca tek bir loop ile uğraşırken etiketler herhangi bir değer katmaz, ancak bir sonraki bölümde öğreneceğiniz gibi, iç içe geçmiş yapılarda son derece faydalıdır.

English: While this topic is not on the exam, it is possible to add optional labels to control and block statements. For example, the following is permitted by the compiler, albeit extremely uncommon: **Türkçe:** Bu konu sınavda yer almamakla birlikte kontrol ve blok ifadelerine isteğe bağlı etiketler eklemek mümkündür. Örneğin, son derece nadir de olsa, derleyici tarafından aşağıdakilere izin verilmektedir:

```
int frog = 15;
```

English: BAD_IDEA: if(frog>10) **Türkçe:** BAD_IDEA: if(frog>10)

EVEN_WORSE_IDEA: {

Türkçe başlık: EVEN_WORSE_IDEA: {

```
frog++;
}
```

The break Statement

Türkçe başlık: break Statement'i

English: As you saw when working with switch statements, a break statement transfers the flow of control out to the enclosing statement. The same holds true for a break statement that appears inside of a while, do/while, or for loop, as it will end the loop early, as shown in Figure 3.9. **Türkçe:** switch statement'larıyla çalışırken gördüğünüz gibi break statement'i control flow'u enclosing statement'in dışına aktarır. Şekil 3.9'da gösterildiği üzere while, do/while veya for loop'u içindeki break statement'i da loop'u erken sonlandırır.

FIGURE 3.9 The structure of a break statement

Türkçe başlık: ŞEKİL 3.9 Bir break statement'inin yapısı

English: Optional reference to head of loop Colon (required if optionalLabel is present) optionalLabel: while(booleanExpression) { **Türkçe:** Loop başına isteğe bağlı reference · optionalLabel varsa iki nokta zorunludur · optionalLabel: while(booleanExpression) {

```
// Body
// Somewhere in the loop
break optionalLabel;
}
```

English: Semicolon (required) break keyword Notice in Figure 3.9 that the break statement can take an optional label parameter. **Türkçe:** Noktalı virgül zorunludur · `break` keyword'ü. Şekil 3.9'da `break` statement'inin isteğe bağlı bir label parameter alabildiğine dikkat edin.

English: Without a label parameter, the break statement will terminate the nearest inner loop it is currently in the process of executing. The optional label parameter allows us to break out of a higher-level outer loop. In the following example, we search for the first (x,y) array index position of a number within an unsorted two-dimensional array: **Türkçe:** Label parameter olmadan `break` statement'ı o anda çalışan en yakın inner loop'u sonlandırır. İsteğe bağlı label parameter, daha üst düzey bir outer loop'tan çıkmamızı sağlar. Aşağıdaki örnekte sıralanmamış iki boyutlu bir array içinde bir sayının ilk (x, y) index konumunu arıyoruz:

```
public class FindInMatrix {
    public static void main(String[] args) {
        int[][] list = {{1,13},{5,2},{2,2}};
        int searchValue = 2;
        int positionX = -1;
        int positionY = -1;
        PARENT_LOOP: for(int i=0; i<list.length; i++) {
            for(int j=0; j<list[i].length; j++) {
                if(list[i][j]==searchValue) {
                    positionX = i;
```

```
                    positionY = j;
                    break PARENT_LOOP;
                }
            }
        }
        if(positionX== -1 || positionY== -1) {
            System.out.println("Value "+searchValue+" not found");
        } else {
            System.out.println("Value "+searchValue+" found at: " +
                "("+positionX+", "+positionY+")");
        }
    }
}
```

English: When executed, this code will output the following: **Türkçe:** Çalıştırıldığında bu kod aşağıdaki çıktıyı verecektir:

English: Value 2 found at: (1,1) In particular, take a look at the statement `break PARENT_LOOP`. This statement will break out of the entire loop structure as soon as the first matching value is found. Now, imagine what would happen if we replaced the body of the inner loop with the following: **Türkçe:** Value 2 found at: (1,1) Özellikle `break PARENT_LOOP` statement'ına bakın. Bu statement ilk eşleşen değer bulunur bulunmaz bütün loop yapısından çıkar. Inner loop'un body kısmını aşağıdakiyle değiştirirsek ne olacağını düşünün:

```
if(list[i][j]==searchValue) {
    positionX = i;
    positionY = j;
    break;
}
```

English: How would this change our flow, and would the output change? Instead of exiting when the first matching value is found, the program would now only exit the inner loop when the condition was met. In other words, the structure would find the first matching value of the last inner loop to contain the value, resulting in the following output: **Türkçe:** Bu değişiklik control flow'u ve output'u nasıl etkiler? Program artık ilk eşleşmede bütün yapıdan çıkmak yerine condition sağlandığında yalnızca inner loop'tan çıkar. Böylece değeri içeren son inner loop'taki ilk eşleşmeyi bulur ve şu output'u üretir:

English: Value 2 found at: (2,0) Finally, what if we removed the break altogether? **Türkçe:** Value 2 found at: (2,0) Son olarak `break` statement'ını tamamen kaldırırsak ne olur?

```
if(list[i][j]==searchValue) {
    positionX = i;
    positionY = j;
}
```

English: In this case, the code would search for the last value in the entire structure that had the matching value. The output would look like this: **Türkçe:** Bu durumda kod, yapının tamamında eşleşen değere sahip olan son değeri arayacaktır. Çıktı şu şekilde görünecektir:

English: Value 2 found at: (2,1) **Türkçe:** Value 2 found at: (2,1)

English: You can see from this example that using a label on a break statement in a nested loop, or not using the break statement at all, can cause the loop structure to behave quite differently. **Türkçe:** Bu örnek, nested loop içindeki bir `break` statement'ında label kullanmanın veya `break` statement'ını hiç kullanmamanın loop yapısının davranışını önemli ölçüde değiştirebildiğini gösterir.

The continue Statement

Türkçe başlık: `continue` Statement'ı

English: Let's now extend our discussion of advanced loop control with the continue statement, a statement that causes flow to finish the execution of the current loop iteration, as shown in Figure 3.10.

Türkçe: Şimdi advanced loop control konusunu, Şekil 3.10'da gösterilen `continue` statement'ıyla genişletelim. Bu statement, control flow'un mevcut loop iteration'ını bitirmesini sağlar.

FIGURE 3.10 The structure of a continue statement

Türkçe başlık: ŞEKİL 3.10 Bir `continue` statement'ının yapısı

English: Optional reference to head of loop Colon (required if optionalLabel is present) optionalLabel: while(booleanExpression) { **Türkçe:** Loop başına isteğe bağlı reference · optionalLabel varsa iki nokta zorunludur · optionalLabel: while(booleanExpression) {

```
// Body
// Somewhere in the loop
continue optionalLabel;
}
```

English: Semicolon (required) continue keyword You may notice that the syntax of the continue statement mirrors that of the break statement. In fact, the statements are identical in how they are used, but with different results. **Türkçe:** Noktalı virgül zorunludur · continue keyword'ü. continue statement'ının syntax'ının break statement'ını yansıttığına dikkat edebilirsiniz. Kullanım biçimleri aynıdır; ancak sonuçları farklıdır.

English: While the break statement transfers control to the enclosing statement, the continue statement transfers control to the boolean expression that determines if the loop should continue. **Türkçe:** break statement'ı kontrolü enclosing statement'ın dışına aktarırken continue statement'ı, loop'un devam edip etmeyeceğini belirleyen boolean expression'a aktarır.

English: In other words, it ends the current iteration of the loop. Also, like the break statement, the continue statement is applied to the nearest inner loop under execution, using optional label statements to override this behavior. **Türkçe:** Başka bir deyişle mevcut loop iteration'ını sona erdirir. break statement'ında olduğu gibi continue statement'ı da varsayılan olarak çalışan en yakın inner loop'a uygulanır; isteğe bağlı label ile bu davranış değiştirilebilir.

English: Let's take a look at an example. Imagine we have a zookeeper who is supposed to clean the first leopard in each of four stables but skip stable b entirely. **Türkçe:** Bir örneğe bakalım. Dört ahırın her birinde ilk leopard'ı temizlemesi gereken ancak ahır b'yi tamamen atlayan bir hayvanat bahçesi bakıcımız olduğunu hayal edin.

```
1: public class CleaningSchedule {
2: public static void main(String[] args) {
3: CLEANING: for(char stables = 'a'; stables<='d'; stables++) {
4: for(int leopard = 1; leopard<4; leopard++) {
5: if(stables=='b' || leopard==2) {
```

```
6: continue CLEANING;
7: }
8: System.out.println("Cleaning: "+stables+", "+leopard);
9: } } } }
```

English: With the structure as defined, the loop will return control to the parent loop any time the first value is b or the second value is 2. On the first, third, and fourth executions of the outer loop, the inner loop prints a statement exactly once and then exits on the next inner loop when leopard is 2. On the second execution of the outer loop, the inner loop immediately exits without printing anything since b is encountered right away. The following is printed: **Türkçe:** Bu yapıda ilk değer **b** veya ikinci değer **2** olduğunda control parent loop'a döner. Outer loop'un birinci, üçüncü ve dördüncü execution'ında inner loop tam bir statement yazdırır; ardından **leopard** değeri **2** olunca sonraki inner loop'tan çıkar. Outer loop'un ikinci execution'ında ise **b** hemen bulunduğu için inner loop hiçbir şey yazdırmadan sona erer. Şu output üretilir:

English: Cleaning: a,1 Cleaning: c,1 Cleaning: d,1 Now, imagine we remove the CLEANING label in the continue statement so that control is returned to the inner loop instead of the outer. Line 6 becomes the following: **Türkçe:** **Cleaning: a,1 Cleaning: c,1 Cleaning: d,1** Şimdi control'ün outer loop yerine inner loop'a dönmesi için **continue** statement'ındaki **CLEANING** label'ını kaldırdığımızı düşünün. 6. satır şu hâle gelir:

```
6: continue;
```

English: This corresponds to the zookeeper cleaning all leopards except those labeled 2 or in stable b. The output is then the following: **Türkçe:** Bu, hayvanat bahçesi görevlisinin 2 veya ahır b olarak etiketlenenler dışındaki tüm leopard'ları temizlemesine karşılık gelir. Çıktı daha sonra aşağıdaki gibidir:

English: Cleaning: a,1 Cleaning: a,3 Cleaning: c,1 Cleaning: c,3 Cleaning: d,1 Cleaning: d,3 Finally, if we remove the continue statement and the associated if statement altogether by removing lines 5-7, we arrive at a structure that outputs all the values, such as this: **Türkçe:** **Cleaning: a,1 Cleaning: a,3 Cleaning: c,1 Cleaning: c,3 Cleaning: d,1 Cleaning: d,3** Son olarak 5-7. satırları kaldırıp **continue** ile ilişkili **if** statement'ını tamamen silersek bütün değerleri yazdıran bir yapıya ulaşırız:

English: Cleaning: a,1 Cleaning: a,2 Cleaning: a,3 Cleaning: b,1 Cleaning: b,2 Cleaning: b,3 Cleaning: c,1 Cleaning: c,2 Cleaning: c,3 Cleaning: d,1 Cleaning: d,2 Cleaning: d,3 **Türkçe:** Cleaning: a,1 Cleaning: a,2 Cleaning: a,3 Cleaning: b,1 Cleaning: b,2 Cleaning: b,3 Cleaning: c,1 Cleaning: c,2 Cleaning: c,3 Cleaning: d,1 Cleaning: d,2 Cleaning: d,3

The return Statement

Türkçe başlık: **return** Statement'ı

English: Given that this book shouldn't be your first foray into programming, we hope you've come across methods that contain return statements. Regardless, we cover how to design and create methods that use them in detail in Chapter 5, "Methods." For now, though, you should be familiar with the idea that creating methods and using return statements can be used as an alternative to using labels and break statements. For example, take a look at this rewrite of our earlier FindInMatrix class: **Türkçe:** Bu kitap programlamayla ilk karşılaşmanız olmamalı; bu nedenle `return` statement'ı içeren method'lar görmüş olmanızı bekliyoruz. Bunları kullanan method'ların nasıl tasarlanıp oluşturulacağını Bölüm 5, "Methods"ta ayrıntılı biçimde ele alıyoruz. Şimdilik method oluşturmanın ve `return` statement'ı kullanmanın label ile `break` statement'larına alternatif olabileceği fikrine aşina olmanız yeterlidir. Örneğin önceki `FindInMatrix` class'ımızın şu yeniden yazımına bakın:

```
public class FindInMatrixUsingReturn {
    private static int[] searchForValue(int[][] list, int v) {
        for (int i = 0; i < list.length; i++) {
            for (int j = 0; j < list[i].length; j++) {
                if (list[i][j] == v) {
                    return new int[] {i,j};
                }
            }
        }
        return null;
    }

    public static void main(String[] args) {
        int[][] list = { { 1, 13 }, { 5, 2 }, { 2, 2 } };
        int searchValue = 2;
        int[] results = searchForValue(list,searchValue);
        if (results == null) {
            System.out.println("Value " + searchValue + " not found");
        } else {
            System.out.println("Value " + searchValue + " found at: " +
                "(" + results[0] + ", " + results[1] + ")");
        }
    }
}
```

English: This class is functionally the same as the first FindInMatrix class we saw earlier using break. If you need finer-grained control of the loop with multiple break and continue statements, the first class is probably better. That said, we find code without labels and break statements a lot easier to read and debug. Also, making the search logic an independent function makes the code more reusable and the calling main() method a lot easier to read. **Türkçe:** Bu class, işlevsel olarak daha önce `break` kullanarak gördüğümüz ilk `FindInMatrix` class'ıyla aynıdır. Birden çok `break` ve `continue` statement'ıyla loop üzerinde daha ayrıntılı kontrole ihtiyacınız varsa ilk class muhtemelen daha uygundur. Bununla birlikte label ve `break` statement'ı içermeyen kodu okumak ve debug etmek çok daha kolaydır. Ayrıca arama mantığını bağımsız bir function haline getirmek kodun yeniden kullanılabilirliğini artırır ve onu çağıran `main()` method'unun okunmasını kolaylaştırır.

English: For the exam, you will need to know both forms. Just remember that return statements can be used to exit loops quickly and can lead to more readable code in practice, especially when used with nested loops. **Türkçe:** Sınav için iki biçimi de bilmelisiniz. `return` statement'larının loop'lardan hızlıca çıkmak için kullanılabildiğini ve özellikle nested loop'larda daha okunabilir kod sağlayabildiğini unutmayın.

Unreachable Code

Türkçe başlık: Unreachable Code (Erişilemeyen Kod)

English: One facet of break, continue, and return that you should be aware of is that any code placed immediately after them in the same block is considered unreachable and will not compile. For example, the following code snippet does not compile: **Türkçe:** break, continue ve return hakkında bilmeniz gereken bir nokta, aynı block'ta bunlardan hemen sonra yer alan kodun unreachable kabul edilmesi ve derlenmemesidir. Örneğin aşağıdaki code snippet derlenmez:

```
int checkDate = 0;
while(checkDate<10) {
    checkDate++;
    if(checkDate>100) {
        break;
        checkDate++; // DOES NOT COMPILE
    }
}
```

English: Even though it is not logically possible for the if statement to evaluate to true in this code sample, the compiler notices that you have statements immediately following the break and will fail to compile with “unreachable code” as the reason. The same is true for continue and return statements, as shown in the following two examples: **Türkçe:** Bu örnekte if statement'ın true olması mantıksal olarak mümkün değildir. Yine de compiler, break statement'ından hemen sonra başka statement'lar bulunduğunu görür ve unreachable code nedeniyle compilation error verir. Aşağıdaki iki örnekte görüldüğü gibi aynı durum continue ve return statement'ları için de geçerlidir:

```
int minute = 1;
WATCH: while(minute>2) {
    if(minute++>2) {
        continue WATCH;
        System.out.print(minute); // DOES NOT COMPILE
    }
}
int hour = 2;
switch(hour) {
    case 1: return; hour++; // DOES NOT COMPILE
    case 2:
    }
```

English: One thing to remember is that it does not matter if the loop or decision structure actually visits the line of code. For example, the loop could execute zero or infinite times at runtime. Regardless of execution, the compiler will report an error if it finds any code it deems unreachable, in this case any statements immediately following a break, continue, or return statement. **Türkçe:** Loop'un veya decision structure'in ilgili kod satırına runtime'da gerçekten ulaşip ulaşmaması önemli değildir; örneğin loop sıfır kez ya da sonsuz kez çalışabilir. Compiler unreachable kabul ettiği bir kod bulursa—bu örnekte break, continue veya return statement'ından hemen sonraki statement'lar—compilation error bildirir.

Summary

Türkçe başlık: Özet

Reviewing Branching

Türkçe başlık: Dallanmanın Gözden Geçirilmesi

English: We conclude this section with Table 3.1, which will help remind you when labels, break, and continue statements are permitted in Java. Although for illustrative purposes our examples use these statements in nested loops, they can be used inside single loops as well. **Türkçe:** Bu bölümü, Java'da label, `break` ve `continue` kullanımına ne zaman izin verildiğini özetleyen Tablo 3.1 ile bitiriyoruz. Örneklerde anlatım amacıyla nested loop'lar kullanılmış olsa da bu statement'lar tek loop içinde de kullanılabilir.

TABLE 3.1 Control statement usage

Türkçe başlık: TABLO 3.1 Kontrol ifadesi kullanımı

Support labels Support break Support continue Support yield

Türkçe başlık: Label desteği · `break` desteği · `continue` desteği · `yield` desteği

while Yes Yes Yes No

Türkçe başlık: `while`: Evet · Evet · Evet · Hayır

do/while Yes Yes Yes No

Türkçe başlık: `do/while`: Evet · Evet · Evet · Hayır

for Yes Yes Yes No

Türkçe başlık: `for`: Evet · Evet · Evet · Hayır

switch Yes Yes No Yes

Türkçe başlık: `switch`: Evet · Evet · Hayır · Evet

English: Last but not least, all testing centers should offer some form of scrap paper or dry-erase board to use during the exam. We strongly recommend you make use of these testing aids, should you encounter complex questions involving nested loops and branching statements. **Türkçe:** Son olarak, tüm sınav merkezleri sınav sırasında kullanılmak üzere bir tür hurda kağıt veya kuru silinebilir tahta sunmalıdır. İç içe loop'lar ve dallanma ifadeleri içeren karmaşık sorularla karşılaşırsanız, bu test yardımcılarından yararlanmanızı önemle tavsiye ederiz.

English: Some of the most time-consuming questions you may see on the exam could involve nested loops with lots of branching. Unless you spot an obvious compiler error, we recommend skipping these questions and coming back to them at the end. Remember, all questions on the exam are weighted evenly! **Türkçe:** Sınavdaki en çok zaman alan soruların bazıları yoğun branching içeren nested loop'larla ilgili olabilir. Açık bir compilation error görmüyorsanız bu soruları önce atlayıp sınav sonunda geri dönmenizi öneririz. Bütün soruların eşit ağırlıkta olduğunu unutmayın.

Summary

Türkçe başlık: Özet

English: This chapter presented how to make intelligent decisions in Java. We covered basic decision-making constructs such as if, else, and switch statements and showed how to use them to change the path of the process at runtime. We also presented newer features in the Java language, including pattern matching and switch expressions, both designed to reduce boilerplate code. **Türkçe:** Bu bölüm Java'da akıllı kararların nasıl verileceğini anlattı. `if`, `else` ve `switch` statement'ları gibi temel decision-making yapılarını ele aldık ve runtime'da execution path'ini değiştirmek için nasıl kullanılacaklarını gösterdik. Ayrıca Java dilindeki yeni özelliklerden pattern matching ve `switch` expression'ları tanıttık; her ikisi de boilerplate code'u azaltmak üzere tasarlanmıştır.

English: We then moved our discussion to repetition control structures, starting with while and do/while loops. We showed how to use them to create processes that loop multiple times and also showed how it is important to make sure they eventually terminate. Remember that most of these structures require the evaluation of a particular boolean expression to complete. **Türkçe:** Ardından `while` ve `do/while` loop'larıyla başlayarak tekrarlı control structure'lara geçtik. Birden çok kez çalışan süreçlerin nasıl kurulacağını ve bu loop'ların sonunda mutlaka sonlanmasının önemini gösterdik. Bu yapıların çoğunun tamamlanması belirli bir boolean expression'ın değerlendirilmesine bağlıdır.

English: Next, we covered the extremely convenient repetition control structures: the for and for-each loops. While their syntax is more complex than the traditional while or do/while loops, they are extremely useful in everyday coding and allow you to create complex expressions in a single line of code. With a for-each loop, you don't need to explicitly write a boolean expression, since the compiler builds one for you. For clarity, we referred to an enhanced for loop as a for-each loop, but syntactically both are written using the for keyword. **Türkçe:** Ardından son derece kullanışlı tekrarlı control structure'ları, `for` ve `for-each` loop'larını ele aldık. Syntax'ları geleneksel `while` veya `do/while` loop'larından daha karmaşık olsa da günlük kodlamada çok kullanışlıdır ve tek satırda karmaşık expression'lar kurmanızı sağlar. `for-each` loop'unda compiler sizin için oluşturduğundan açık bir boolean expression yazmanız gerekmez. Açıklık için enhanced `for` loop'una `for-each` loop'u dedik; syntax bakımından ikisi de `for` keyword'üyle yazılır.

English: We concluded this chapter by discussing advanced control options and how flow can be enhanced through nested loops coupled with break, continue, and return statements. Be wary of questions on the exam that use nested loops, especially ones with labels, and verify that they are being used correctly. **Türkçe:** Bölümü advanced control seçeneklerini ve nested loop'larla birlikte kullanılan `break`, `continue` ve `return` statement'larının akışı nasıl değiştirdiğini ele alarak bitirdik. Sınavda nested loop, özellikle label kullanan sorulara dikkat edin ve yapıların doğru kullanıldığını doğrulayın.

English: This chapter is especially important because at least one component of this chapter will likely appear in every exam question with sample code. Many of the questions on the exam focus on proper syntactic use of the structures, as they will be a large source of questions that end in “Does not compile.” You should be able to answer all of the review questions correctly or fully understand those that you answered incorrectly before moving on to later chapters. **Türkçe:** Bu bölüm özellikle önemlidir çünkü bu bölümün en az bir bileşeni muhtemelen her sınav sorusunda örnek kodla birlikte görünecektir. Sınavdaki soruların çoğu, "Derlenmiyor" ile biten soruların büyük bir kaynağı olacağından, yapıların sözdizimsel olarak doğru kullanımına odaklanıyor. Sonraki bölümlere geçmeden önce tüm inceleme sorularını doğru yanıtlayabilmeli veya yanlış yanıtladığınız soruları tam olarak anlayabilmelisiniz.

Exam Essentials

Türkçe başlık: Sınav Esasları

English: Understand if and else decision control statements. The if and else statements come up frequently throughout the exam in questions unrelated to decision control, so make sure you fully understand these basic building blocks of Java. **Türkçe:** `if` ve `else` decision-control statement'larını anlayın. Bu yapılar, sınav boyunca decision control ile doğrudan ilgili olmayan sorularda da sıkça karşınıza çıkar; Java'nın bu temel yapı taşlarını tam olarak öğrenin.

English: Apply pattern matching and flow scoping. Pattern matching can be used to reduce boilerplate code involving an if statement, instanceof operator, and cast operation using a pattern variable. It can also include a pattern or filter after the pattern variable declaration. **Türkçe:** Pattern matching ve flow scoping uygulayın. Pattern matching; `if` statement'ı, `instanceof` operator'ü ve pattern variable ile cast işlemi içeren boilerplate code'u azaltabilir. Pattern variable bildiriminden sonra ek bir pattern veya filtre de içerebilir.

English: Pattern matching uses flow scoping in which the pattern variable is in scope as long as the compiler can definitively determine its type. **Türkçe:** Pattern matching flow scoping kullanır: Compiler pattern variable'ın type'ını kesin olarak belirleyebildiği sürece variable scope içindedir.

English: Understand switch statements and their proper usage. You should be able to spot a poorly formed switch statement on the exam. The switch value and data type should be compatible with the case statements, and the values for the case statements must evaluate to compile-time constants. Finally, at runtime, a switch statement branches to the first matching case, or default if there is no match, or exits entirely if there is no match and no default branch. The process then continues into any proceeding case or default statements until a break or return statement is reached. **Türkçe:** `switch` statement'larını ve doğru kullanımlarını anlayın. Sınavda hatalı oluşturulmuş bir `switch` statement'ını fark edebilmelisiniz. Switch value ile data type, `case` statement'larıyla uyumlu olmalı; `case` değerleri compile-time constant olmalıdır. Runtime'da akış ilk eşleşen `case`ee, eşleşme yoksa `default`a gider; ikisi de yoksa `switch` sona erer. Ardından `break` veya `return` statement'ına ulaşılan kadar izleyen `case` ya da `default` statement'ları çalışır.

English: Use switch expressions correctly. Discern the differences between switch expressions and switch statements. Understand how to write switch expressions correctly, including proper use of semicolons, writing case expressions and blocks that yield a consistent value, and making sure all possible values of the switch variable are handled by the switch expression. **Türkçe:** `switch` expression'ları doğru kullanın. `switch` expression ile `switch` statement arasındaki farkları ayırt edin. Noktalı virgüllerin doğru kullanımı, tutarlı değer üreten `case` expression ve block'larının yazılması ve switch variable'ın bütün olası değerlerinin işlenmesi dahil `switch` expression'ın nasıl doğru yazılacağını anlayın.

Exam Essentials

Türkçe başlık: Sınav Esasları

English: Write while loops. Know the syntactical structure of all while and do/while loops. In particular, know when to use one versus the other. **Türkçe:** `while` loop'ları yazın. Bütün `while` ve `do/while` loop'larının syntax yapısını ve özellikle hangisinin ne zaman kullanılacağını öğrenin.

English: Be able to use for loops. You should be familiar with for and for-each loops and know how to write and evaluate them. Each loop has its own special properties and structures. **Türkçe:** `for` loop'larını kullanabilin. `for` ve `for-each` loop'larını tanıyın; bunların nasıl yazılıp değerlendirileceğini bilin. Her loop'un kendine özgü özellikleri ve yapısı vardır.

English: You should know how to use for-each loops to iterate over lists and arrays. **Türkçe:** List'ler ve array'ler üzerinde iteration yapmak için `for-each` loop'unun nasıl kullanılacağını bilmelisiniz.

English: Understand how break, continue, and return can change flow control. Know how to change the flow control within a statement by applying a break, continue, or return statement. Also know which control statements can accept break statements and which can accept continue statements. Finally, you should understand how these statements work inside embedded loops or switch statements. **Türkçe:** `break`, `continue` ve `return` statement'larının flow control'ü nasıl değiştirdiğini anlayın. Bu statement'ları uygulayarak bir yapıdaki control flow'un nasıl değiştirileceğini; hangi control statement'ların `break`, hangilerinin `continue` kabul ettiğini öğrenin. Son olarak bunların embedded loop'lar ve `switch` statement'ları içinde nasıl çalıştığını anlayın.

Review Questions

Türkçe başlık: Review Questions

English: The answers to the chapter review questions can be found in the Appendix. **Türkçe:** Bölüm inceleme sorularının yanıtlarını Ek'te bulabilirsiniz.

English: 1. Which of the following data types can be used in a switch expression? (Choose all that apply.) **Türkçe:** 1. Aşağıdaki veri türlerinden hangisi bir `switch` expression'ında kullanılabilir? (Geçerli olanların tümünü seçin.)

English: A. enum **Türkçe:** A. enum

B. int

C. Byte

Türkçe başlık: C. Byte

D. long
E. String
F. char
G. var
H. double

English: 2. What is the output of the following code snippet? (Choose all that apply.) **Türkçe:** 2. Aşağıdaki kod parçacığının çıktısı nedir? (Geçerli olanların tümünü seçin.)

```
3: int temperature = 4;  
4: long humidity = -temperature + temperature * 3;  
5: if (temperature >= 4)  
6: if (humidity < 6) System.out.println("Too Low");  
7: else System.out.println("Just Right");  
8: else System.out.println("Too High");
```

A. Too Low

Türkçe başlık: A. Too Low

B. Just Right

Türkçe başlık: B. Just Right

C. Too High

Türkçe başlık: C. Too High

English: D. A NullPointerException is thrown at runtime. **Türkçe:** D. runtime'da bir NullPointerException atılır.

English: E. The code will not compile because of line 7. **Türkçe:** E. Kod line 7 nedeniyle derlenmez.

English: F. The code will not compile because of line 8. **Türkçe:** F. Kod line 8 nedeniyle derlenmez.

English: 3. Which of the following data types are permitted on the right side of a for-each expression? **Türkçe:** 3. `for-each` statement'inın sağ tarafında aşağıdaki data type'lardan hangilerine izin verilir?

English: (Choose all that apply.) **Türkçe:** (Geçerli olanların tümünü seçin.)

A. Double[][]

Türkçe başlık: A. Double[][]

B. Object

Türkçe başlık: B. Object

C. Map

Türkçe başlık: C. Map

D. List

Türkçe başlık: D. List

E. String
F. char[]

G. Exception

Türkçe başlık: G. Exception

H. Set

Türkçe başlık: H. Set

Questions 4 and 5

Türkçe başlık: Soru 4 ve 5

English: 4. What is the output of calling printReptile(6)? **Türkçe:** 4. printReptile(6)'yı çağırmanın çıktısı nedir?

```
void printReptile(int category) {  
    var type = switch(category) {  
        case 1,2 -> "Snake";  
        case 3,4 -> "Lizard";  
        case 5,6 -> "Turtle";  
        case 7,8 -> "Alligator";  
    };  
    System.out.print(type);  
}
```

A. Snake**Türkçe başlık:** A. Snake**B. Lizard****Türkçe başlık:** B. Lizard**C. Turtle****Türkçe başlık:** C. Turtle**D. Alligator****Türkçe başlık:** D. Alligator**E. TurtleAlligator****Türkçe başlık:** E. TurtleAlligator**English:** F. None of the above **Türkçe:** F. Yukarıdakilerin hiçbiri**English:** 5. What is the output of the following code snippet? **Türkçe:** 5. Aşağıdaki kod parçasığının çıktısı nedir?

```
List<Integer> myFavoriteNumbers = new ArrayList<>();
myFavoriteNumbers.add(10);
myFavoriteNumbers.add(14);
for (var a : myFavoriteNumbers) {
    System.out.print(a + ", ");
    break;
}
for (int b : myFavoriteNumbers) {
    continue;
    System.out.print(b + ", ");
}
for (Object c : myFavoriteNumbers)
    System.out.print(c + ", ");
```

English: A. It compiles and runs without issue but does not produce any output. **Türkçe:** A. Kod sorunsuz derlenip çalışır; ancak output üretmez.**English:** B. 10, 14, **Türkçe:** B. 10, 14,**English:** C. 10, 10, 14, **Türkçe:** C. 10, 10, 14,

English: D. 10, 10, 14, 10, 14, **Türkçe:** D. 10, 10, 14, 10, 14,

English: E. Exactly one line of code does not compile. **Türkçe:** E. Kodun tam olarak bir satırı derlenmez.

English: F. Exactly two lines of code do not compile. **Türkçe:** F. Kodun tam olarak iki satırı derlenmez.

English: G. Three or more lines of code do not compile. **Türkçe:** G. Kodun üç veya daha fazla satırı derlenmez.

English: H. The code contains an infinite loop and does not terminate. **Türkçe:** H. Kod sonsuz bir loop içerir ve sona ermez.

English: 6. Which statements about decision structures are true? (Choose all that apply.) **Türkçe:** 6. Decision structure'larla ilgili hangi statement'lar doğrudur? (Geçerli olanların tümünü seçin.)

English: A. A for-each loop can be executed on any Collections Framework object. **Türkçe:** A. Bir `for-each` loop'u, herhangi bir Collections Framework nesnesi üzerinde çalıştırılabilir.

English: B. The body of a while loop is guaranteed to be executed at least once. **Türkçe:** B. Bir `while` loop'unun gövdesinin en az bir kez çalıştırılması garanti edilir.

English: C. The conditional expression of a for loop is evaluated before the first execution of the loop body. **Türkçe:** C. Bir `for` loop'unun conditional expression'ı, loop gövdesi ilk kez çalıştırılmadan önce değerlendirilir.

English: D. A switch expression that takes a String and assigns the result to a variable requires a default branch. **Türkçe:** D. Bir `String` alıp sonucu bir variable'a atayan `switch` expression, bir `default` branch gerektirir.

English: E. The body of a do/while loop is guaranteed to be executed at least once. **Türkçe:** E. Bir `do/while` loop'unun gövdesinin en az bir kez çalıştırılması garanti edilir.

English: F. An if statement can have multiple corresponding else statements. **Türkçe:** F. Bir `if` statement'ın birden fazla karşılık gelen `else` statement'ı olabilir.

English: 7. Assuming weather is a well-formed nonempty array, which code snippet, when inserted independently into the blank in the following code, prints all of the elements of weather? **Türkçe:** 7. `weather`ın düzgün oluşturulmuş, boş olmayan bir array olduğunu varsayarsak aşağıdaki boşluğa bağımsız olarak yerleştirilen hangi code snippet'leri `weather`ın bütün element'lerini yazdırır?

English: (Choose all that apply.) **Türkçe:** (Geçerli olanların tümünü seçin.)

```
private void print(int[] weather) {  
    for( ) {  
        System.out.println(weather[i]);  
    }  
}  
A. int i=weather.length; i>0; i--
```

English: B. **Türkçe:** B.

```
int i=0; i<=weather.length-1; ++i  
C. var w : weather  
D. int i=weather.length-1; i>=0; i--
```

English: E. **Türkçe:** E.

```
int i=0, int j=3; i<weather.length; ++i  
F. int i=0; ++i<10 && i<weather.length;
```

English: G. None of the above **Türkçe:** G. Yukarıdakilerin hiçbiri

English: 8. What is the output of calling printType(11)? **Türkçe:** 8. printType(11)'i çağırmanın çıktısı nedir?

```
31: void printType(Object o) {  
32: if(o instanceof Integer bat) {  
33: System.out.print("int");  
34: } else if(o instanceof Integer bat && bat < 10) {  
35: System.out.print("small int");  
36: } else if(o instanceof Long bat || bat <= 20) {  
37: System.out.print("long");  
38: } default {  
39: System.out.print("unknown");  
40: }  
41: }
```

A. int

English: B. small int **Türkçe:** B. small int

C. long

English: D. unknown **Türkçe:** D. unknown

English: E. Nothing is printed. **Türkçe:** E. Hiçbir şey basılmıyor.

English: F. The code contains one line that does not compile. **Türkçe:** F. Kod derlenmeyen bir satır içeriyor.

English: G. The code contains two lines that do not compile. **Türkçe:** G. Kod derlenmeyen iki satır içeriyor.

English: H. None of the above **Türkçe:** H. Yukarıdakilerin hiçbiri

English: 9. Which statements, when inserted independently into the following blank, will cause the code to print 2 at runtime? (Choose all that apply.) **Türkçe:** 9. Hangi ifadeler aşağıdaki boşluğa bağımsız olarak eklendiğinde kodun runtime'da 2 basmasına neden olur? (Geçerli olanların tümünü seçin.)

```
int count = 0;
BUNNY: for(int row = 1; row <=3; row++)
RABBIT: for(int col = 0; col <3 ; col++) {
    if((col + row) % 2 == 0)
    ;
    count++;
}
System.out.println(count);
```

A. break BUNNY

Türkçe başlık: A. break BUNNY

B. break RABBIT

Türkçe başlık: B. break RABBIT

C. continue BUNNY

Türkçe başlık: C. continue BUNNY

D. continue RABBIT

Türkçe başlık: D. continue RABBIT

E. break

Türkçe başlık: E. break

F. continue

Türkçe başlık: F. continue

English: G. None of the above, as the code contains a compiler error. **Türkçe:** G. Kod bir derleyici hatası içerdiğinden yukarıdakilerin hiçbiri.

English: 10. Given the following method, how many lines contain compilation errors? (Choose all that apply.) **Türkçe:** 10. Aşağıdaki method göz önüne alındığında, kaç satırda derleme hatası var? (Geçerli olanların tümünü seçin.)

```
10: private DayOfWeek getWeekDay(int day, final int thursday) {  
11: int otherDay = day;  
12: int Sunday = 0;  
13: switch(otherDay) {  
14: default:  
15: case 1: continue;  
16: case thursday: return DayOfWeek.THURSDAY;  
17: case 2,10: break;
```

```
18: case Sunday: return DayOfWeek.SUNDAY;  
19: case DayOfWeek.MONDAY: return DayOfWeek.MONDAY;  
20: }  
21: return DayOfWeek.FRIDAY;  
22: }
```

English: A. None, the code compiles without issue. **Türkçe:** A. Hiçbiri; kod sorunsuz biçimde derlenir.

English: B. 1 **Türkçe:** B. 1

English: C. 2 **Türkçe:** C. 2

English: D. 3 **Türkçe:** D. 3

English: E. 4 **Türkçe:** E. 4

English: F. 5 **Türkçe:** F. 5

English: G. 6 **Türkçe:** G. 6

English: H. The code compiles but may produce an error at runtime. **Türkçe:** H. Kod derleniyor ancak runtime'da hata üretebilir.

English: 11. What is the output of calling `printLocation(Animal.MAMMAL)`? **Türkçe:** 11. `printLocation(Animal.MAMMAL)` çağrısının output'u nedir?

```
10: class Zoo {
11:   enum Animal {BIRD, FISH, MAMMAL}
12:   void printLocation(Animal a) {
13:     long type = switch(a) {
14:       case BIRD -> 1;
15:       case FISH -> 2;
16:       case MAMMAL -> 3;
17:       default -> 4;
18:     };
19:     System.out.print(type);
20:   } }
```

English: A. 3 **Türkçe:** A. 3

English: B. 4 **Türkçe:** B. 4

English: C. 34 **Türkçe:** C. 34

English: D. The code does not compile because of line 13. **Türkçe:** D. Kod 13. satır nedeniyle derlenmez.

English: E. The code does not compile because of line 17. **Türkçe:** E. Kod 17. satır nedeniyle derlenmez.

English: F. None of the above **Türkçe:** F. Yukarıdakilerin hiçbiri

English: 12. What is the result of the following code snippet? **Türkçe:** 12. Aşağıdaki kod parçacığının sonucu nedir?

```
3: int sing = 8, squawk = 2, notes = 0;
4: while(sing > squawk) {
5:   sing--;
6:   squawk += 2;
```

```
7:   notes += sing + squawk;
8: }
9: System.out.println(notes);
```

English: A. 11 **Türkçe:** A. 11

English: B. 13 **Türkçe:** B. 13

English: C. 23 **Türkçe:** C. 23

English: D. 33 **Türkçe:** D. 33

English: E. 50 **Türkçe:** E. 50

English: F. The code will not compile because of line 7. **Türkçe:** F. Kod line 7 nedeniyle derlenmez.

English: 13. What is the output of the following code snippet? **Türkçe:** 13. Aşağıdaki kod parçasığının çıktısı nedir?

```
2: boolean keepGoing = true;
3: int result = 15, meters = 10;
4: do {
5:     meters--;
6:     if(meters==8) keepGoing = false;
7:     result -= 2;
8: } while keepGoing;
9: System.out.println(result);
```

English: A. 7 **Türkçe:** A. 7

English: B. 9 **Türkçe:** B. 9

English: C. 10 **Türkçe:** C. 10

English: D. 11 **Türkçe:** D. 11

English: E. 15 **Türkçe:** E. 15

English: F. The code will not compile because of line 6. **Türkçe:** F. Kod 6. satır nedeniyle derlenmez.

English: G. The code does not compile for a different reason. **Türkçe:** G. Kod farklı bir nedenle derlenmez.

English: 14. Which statements about the following code snippet are correct? (Choose all that apply.)
Türkçe: 14. Aşağıdaki kod parçasığına ilişkin hangi ifadeler doğrudur? (Geçerli olanların tümünü seçin.)

```
for(var penguin : new int[2])
System.out.println(penguin);
var ostrich = new Character[3];
for(var emu : ostrich)
System.out.println(emu);
List<Integer> parrots = new ArrayList<Integer>();
for(var macaw : parrots)
System.out.println(macaw);
```

English: A. The data type of penguin is Integer. **Türkçe:** A. `penguin` variable'nın data type'i `Integer`dir.

English: B. The data type of penguin is int. **Türkçe:** B. `penguin` variable'nın data type'i `int`tir.

English: C. The data type of emu is undefined. **Türkçe:** C. `emu` variable'ının data type'ı tanımsızdır.

English: D. The data type of emu is Character. **Türkçe:** D. `emu` variable'ının data type'ı `Character`dir.

English: E. The data type of macaw is List. **Türkçe:** E. `macaw` variable'ının data type'ı `List`dir.

English: F. The data type of macaw is Integer. **Türkçe:** F. `macaw` variable'ının data type'ı `Integer`dir.

English: G. None of the above, as the code does not compile. **Türkçe:** G. Kod derlenmediği için yukarıdakilerin hiçbiri.

English: 15. What is the result of the following code snippet? **Türkçe:** 15. Aşağıdaki kod parçacığının sonucu nedir?

```
final char a = 'A', e = 'E';
char grade = 'B';
switch (grade) {
default:
case a:
case 'B': 'C': System.out.print("great ");
case 'D': System.out.print("good "); break;
case e:
case 'F': System.out.print("not good ");
}
```

English: A. great **Türkçe:** A. great

English: B. great good **Türkçe:** B. great good

English: C. good **Türkçe:** C. good

English: D. not good **Türkçe:** D. not good

English: E. The code does not compile because the data type of one or more case statements does not match the data type of the switch variable. **Türkçe:** E. Bir veya daha fazla `case` statement'in data type'ı `switch` variable'ının data type'ıyla eşleşmediği için kod derlenmez.

English: F. None of the above **Türkçe:** F. Yukarıdakilerin hiçbiri

English: 16. Given the following array, which code snippets print the elements in reverse order from how they are declared? (Choose all that apply.) **Türkçe:** 16. Aşağıdaki array göz önüne alındığında, hangi kod parçacıkları öğeleri bildirilme biçimlerinin tersi sırayla yazdırır? (Geçerli olanların tümünü seçin.)

```
char[] wolf = {'W', 'e', 'b', 'b', 'y'};
```

English: A. Türkçe: A.

```
int q = wolf.length;
for( ; ; ) {
    System.out.print(wolf[--q]);
    if(q==0) break;
}
```

English: B. Türkçe: B.

```
for(int m=wolf.length-1; m>=0; --m)
    System.out.print(wolf[m]);
```

English: C. Türkçe: C.

```
for(int z=0; z<wolf.length; z++)
    System.out.print(wolf[wolf.length-z]);
```

English: D. Türkçe: D.

```
int x = wolf.length-1;
for(int j=0; x>=0 && j==0; x--)
    System.out.print(wolf[x]);
```

English: E. Türkçe: E.

```
final int r = wolf.length;
for(int w = r-1; r>-1; w = r-1)
    System.out.print(wolf[w]);
```

English: F. Türkçe: F.

```
for(int i=wolf.length; i>0; --i)
    System.out.print(wolf[i]);
```

English: G. None of the above Türkçe: G. Yukarıdakilerin hiçbiri

English: 17. What distinct numbers are printed when the following method is executed? (Choose all that apply.) Türkçe: 17. Aşağıdaki method çalıştırıldığında hangi farklı sayılar yazdırılır? (Geçerli olanların tümünü seçin.)

```
private void countAttendees() {  
    int participants = 4, animals = 2, performers = -1;  
    while((participants = participants+1) < 10) {}  
    do {} while (animals++ <= 1);  
    for( ; performers<2; performers+=2) {}  
    System.out.println(participants);  
    System.out.println(animals);  
    System.out.println(performers);  
}
```

English: A. 6 **Türkçe:** A. 6

English: B. 3 **Türkçe:** B. 3

English: C. 4 **Türkçe:** C. 4

English: D. 5 **Türkçe:** D. 5

English: E. 10 **Türkçe:** E. 10

English: F. 9 **Türkçe:** F. 9

English: G. The code does not compile. **Türkçe:** G. Kod derlenmez.

English: H. None of the above **Türkçe:** H. Yukarıdakilerin hiçbiri

English: 18. Which statements about pattern matching and flow scoping are correct? (Choose all that apply.) **Türkçe:** 18. Pattern matching ve flow scoping hakkında hangi statement'lar doğrudur? (Geçerli olanların tümünü seçin.)

English: A. Pattern matching with an if statement is implemented using the instance operator. **Türkçe:** A. Bir `if` statement'ıyla pattern matching, `instance` operator'ü kullanılarak gerçekleştirilir.

English: B. Pattern matching with an if statement is implemented using the instanceon operator. **Türkçe:** B. Bir `if` statement'ıyla pattern matching, `instanceon` operator'ü kullanılarak gerçekleştirilir.

English: C. Pattern matching with an if statement is implemented using the instanceof operator. **Türkçe:** C. Bir `if` statement'ıyla pattern matching, `instanceof` operator'ü kullanılarak gerçekleştirilir.

English: D. The pattern variable cannot be accessed after the if statement in which it is declared. **Türkçe:** D. Pattern variable'a, bildirildiği `if` statement'ından sonra erişilemez.

English: E. Flow scoping means a pattern variable is only accessible if the compiler can discern its type. **Türkçe:** E. Flow scoping, bir pattern variable'a yalnızca compiler onun type'ını kesin olarak belirleyebiliyorsa erişilebileceği anlamına gelir.

English: F. Pattern matching can be used to declare a variable with an else statement. **Türkçe:** F. Pattern matching, `else` statement'ıyla bir variable bildirmek için kullanılabilir.

English: 19. What is the output of the following code snippet? **Türkçe:** 19. Aşağıdaki kod parçasının çıktısı nedir?

```
2: double iguana = 0;
3: do {
4:   int snake = 1;
5:   System.out.print(snake++ + " ");
6:   iguana--;
7: } while (snake <= 5);
8: System.out.println(iguana);
```

English: A. 1 2 3 4 -4.0 **Türkçe:** A. 1 2 3 4 -4.0

English: B. 1 2 3 4 -5.0 **Türkçe:** B. 1 2 3 4 -5.0

English: C. 1 2 3 4 5 -4.0 **Türkçe:** C. 1 2 3 4 5 -4.0

English: D. 0 1 2 3 4 5 -5.0 **Türkçe:** D. 0 1 2 3 4 5 -5.0

English: E. The code does not compile. **Türkçe:** E. Kod derlenmez.

English: F. The code compiles but produces an infinite loop at runtime. **Türkçe:** F. Kod derlenir ancak runtime'da sonsuz bir loop'a girer.

English: G. None of the above **Türkçe:** G. Yukarıdakilerin hiçbirisi

English: 20. Which statements, when inserted into the following blanks, allow the code to compile and run without entering an infinite loop? (Choose all that apply.) **Türkçe:** 20. Hangi ifadeler aşağıdaki boşluklara eklendiğinde kodun sonsuz bir loop'a girmeden derlenmesine ve çalıştırılmasına olanak tanır? (Geçerli olanların tümünü seçin.)

```
4: int height = 1;
5: L1: while(height++ <10) {
6: long humidity = 12;
7: L2: do {
8: if(humidity-- % 12 == 0) ;
9: int temperature = 30;
10: L3: for( ; ; ) {
11: temperature++;
12: if(temperature>50) ;
13: }
14: } while (humidity > 4);
15: }
```

English: A. break L2 on line 8; continue L2 on line 12 **Türkçe:** A. 8. satırda `break L2`; 12. satırda `continue L2`

English: B. continue on line 8; continue on line 12 **Türkçe:** B. 8. satırda `continue`; 12. satırda `continue`

English: C. break L3 on line 8; break L1 on line 12 **Türkçe:** C. 8. satırda `break L3`; 12. satırda `break L1`

English: D. continue L2 on line 8; continue L3 on line 12 **Türkçe:** D. 8. satırda `continue L2`; 12. satırda `continue L3`

English: E. continue L2 on line 8; continue L2 on line 12 **Türkçe:** E. 8. satırda `continue L2`; 12. satırda `continue L2`

English: F. None of the above, as the code contains a compiler error **Türkçe:** F. Kod bir derleyici hatası içerdiğinden yukarıdakilerin hiçbirisi

English: 21. A minimum of how many lines need to be corrected before the following method will compile? **Türkçe:** 21. Aşağıdaki method'un derlenmesi için en az kaç satırın düzeltilmesi gerekir?

```
21: void findZookeeper(Long id) {
22: System.out.print(switch(id) {
23: case 10 -> {"Jane"}
24: case 20 -> {yield "Lisa";};
25: case 30 -> "Kelly";
26: case 30 -> "Sarah";
27: default -> "Unassigned";
28: });
29: }
```

A. Zero

Türkçe başlık: A. Zero

B. One**Türkçe başlık:** B. One**C. Two****Türkçe başlık:** C. Two**D. Three****Türkçe başlık:** D. Three**E. Four****Türkçe başlık:** E. Four**F. Five****Türkçe başlık:** F. Five

English: 22. What is the output of the following code snippet? (Choose all that apply.) **Türkçe:** 22. Aşağıdaki kod parçacığının çıktısı nedir? (Geçerli olanların tümünü seçin.)

```
2: var tailFeathers = 3;
3: final var one = 1;
4: switch (tailFeathers) {
5: case one: System.out.print(3 + " ");
6: default: case 3: System.out.print(5 + " ");
7: }
8: while (tailFeathers > 1) {
9: System.out.print(--tailFeathers + " "); }
```

English: A. 3 **Türkçe:** A. 3**English:** B. 5 1 **Türkçe:** B. 5 1**English:** C. 5 2 **Türkçe:** C. 5 2**English:** D. 3 5 1 **Türkçe:** D. 3 5 1**English:** E. 5 2 1 **Türkçe:** E. 5 2 1

English: F. The code will not compile because of lines 3-5. **Türkçe:** F. Kod 3-5. satırlar nedeniyle derlenmez.

English: G. The code will not compile because of line 6. **Türkçe:** G. Kod 6. satır nedeniyle derlenmez.

English: 23. What is the output of the following code snippet? **Türkçe:** 23. Aşağıdaki kod parçasının çıktısı nedir?

```
15: int penguin = 50, turtle = 75;
16: boolean older = penguin >= turtle;
17: if (older = true) System.out.println("Success");
18: else System.out.println("Failure");
19: else if (penguin != 50) System.out.println("Other");
```

A. Success

Türkçe başlık: A. Success

B. Failure

Türkçe başlık: B. Failure

C. Other

Türkçe başlık: C. Other

English: D. The code will not compile because of line 17. **Türkçe:** D. Kod 17. satır nedeniyle derlenmez.

English: E. The code compiles but throws an exception at runtime. **Türkçe:** E. Kod derlenir ancak runtime'da bir exception fırlatır.

English: F. None of the above **Türkçe:** F. Yukarıdakilerin hiçbiri

English: 24. Which of the following are possible data types for friends that would allow the code to compile? (Choose all that apply.) **Türkçe:** 24. Kodun derlenmesini sağlayacak `friends` data type'ları hangileridir? (Geçerli olanların tümünü seçin.)

```
for (var friend in friends) {
    System.out.println(friend);
}
```

A. Set

Türkçe başlık: A. Set

B. Map**Türkçe başlık:** B. Map

C. String
D. int[]

E. Collection**Türkçe başlık:** E. Collection**F. StringBuilder****Türkçe başlık:** F. StringBuilder**English:** G. None of the above **Türkçe:** G. Yukarıdakilerin hiçbiri**English:** 25. What is the output of the following code snippet? **Türkçe:** 25. Aşağıdaki kod parçasının çıktısı nedir?

```
6: String instrument = "violin";  
7: final String CELLO = "cello";  
8: String viola = "viola";  
9: int p = -1;  
10: switch(instrument) {  
11: case "bass" : break;  
12: case CELLO : p++;  
13: default: p++;  
14: case "VIOLIN": p++;  
15: case "viola" : ++p; break;  
16: }  
17: System.out.print(p);
```

English: A. -1 **Türkçe:** A. -1**English:** B. 0 **Türkçe:** B. 0**English:** C. 1 **Türkçe:** C. 1**English:** D. 2 **Türkçe:** D. 2**English:** E. 3 **Türkçe:** E. 3**English:** F. The code does not compile. **Türkçe:** F. Kod derlenmez.

English: 26. What is the output of the following code snippet? (Choose all that apply.) **Türkçe:** 26. Aşağıdaki kod parçasığının çıktısı nedir? (Geçerli olanların tümünü seçin.)

```
9: int w = 0, r = 1;
10: String name = "";
11: while(w < 2) {
12:     name += "A";
13:     do {
14:         name += "B";
15:         if(name.length()>0) name += "C";
16:     } else break;
17: } while (r <=1);
18: r++; w++; }
19: System.out.println(name);
```

A. ABC

Türkçe başlık: A. ABC

B. ABCABC

Türkçe başlık: B. ABCABC

C. ABCABCABC

Türkçe başlık: C. ABCABCABC

English: D. Line 15 contains a compilation error. **Türkçe:** D. 15. satır bir compilation error içerir.

English: E. Line 18 contains a compilation error. **Türkçe:** E. 18. satır bir compilation error içerir.

English: F. The code compiles but never terminates at runtime. **Türkçe:** F. Kod derlenir ancak runtime'da hiçbir zaman sona ermez.

English: G. The code compiles but throws a `NullPointerException` at runtime. **Türkçe:** G. Kod derlenir ancak runtime'da bir `NullPointerException` fırlatır.

English: 27. What is printed by the following code snippet? **Türkçe:** 27. Aşağıdaki kod parçasığında ne yazdırılıyor?

```
23: byte amphibian = 1;
24: String name = "Frog";
25: String color = switch(amphibian) {
26: case 1 -> { yield "Red"; }
27: case 2 -> { if(name.equals("Frog")) yield "Green"; }
28: case 3 -> { yield "Purple"; }
29: default -> throw new RuntimeException();
30: };
31: System.out.print(color);
```

A. Red

Türkçe başlık: A. Red

B. Green

Türkçe başlık: B. Green

C. Purple

Türkçe başlık: C. Purple

D. RedPurple

Türkçe başlık: D. RedPurple

English: E. An exception is thrown at runtime. **Türkçe:** E. Runtime'da bir exception fırlatılır.

English: F. The code does not compile. **Türkçe:** F. Kod derlenmez.

English: 28. What is the output of calling getFish("goldie")? **Türkçe:** 28. getFish("goldie") çağrısının çıktısı nedir?

```
40: void getFish(Object fish) {
41: if (!(fish instanceof String guppy))
42: System.out.print("Eat!");
43: else if (!(fish instanceof String guppy)) {
44: throw new RuntimeException();
45: }
46: System.out.print("Swim!");
47: }
```

English: A. Eat! **Türkçe:** A. Eat!

English: B. Swim! **Türkçe:** B. Swim!

English: C. Eat! followed by an exception. **Türkçe:** C. **Eat!** yazdırılır; ardından bir exception atılır.

English: D. Eat!Swim! **Türkçe:** D. Eat!Swim!

English: E. An exception is printed. **Türkçe:** E. Bir exception yazdırılır.

English: F. None of the above **Türkçe:** F. Yukarıdakilerin hiçbiri

English: 29. What is the result of the following code? **Türkçe:** 29. Aşağıdaki kodun sonucu nedir?

```
1: public class PrintIntegers {  
2: public static void main(String[] args) {  
3: int y = -2;  
4: do System.out.print(++y + " ");  
5: while(y <= 5);  
6: } }
```

English: A. -2 -1 0 1 2 3 4 5 **Türkçe:** A. -2 -1 0 1 2 3 4 5

English: B. -2 -1 0 1 2 3 4 **Türkçe:** B. -2 -1 0 1 2 3 4

English: C. -1 0 1 2 3 4 5 6 **Türkçe:** C. -1 0 1 2 3 4 5 6

English: D. -1 0 1 2 3 4 5 **Türkçe:** D. -1 0 1 2 3 4 5

English: E. The code will not compile because of line 5. **Türkçe:** E. Kod 5. satır nedeniyle derlenmez.

English: F. The code contains an infinite loop and does not terminate. **Türkçe:** F. Kod sonsuz bir loop içerir ve sona ermez.

Appendix · Önceki çözüm ve teknik pekiştirme notları (kaynak dışı)

Bu appendix önceki kullanıcı çalışmasını kaybetmemek için korunmuştur. Aşağıdaki cevaplar ve hafıza kartları kaynak bölümün birebir çevirisi değildir.

Unit 03 · Making Decisions · Bilingual Notes

Bu ana kaynak gönderilen Chapter 3 Review Questions 12–29 metnini sırasıyla işler. Her English soru özeti ve Türkçe karşılığı art arda gelir. Ayrıntılı dil çalışması için [vocabulary](#) ve [grammar notes](#) dosyalarını kullan.

İçindekiler

1 Question 12 · Partial source

- 2 Questions 13–18 · Loops, switch and pattern matching
- 3 Questions 19–24 · Scope, labels and switch expressions
- 4 Questions 25–29 · Fall-through, infinite loops and flow scope
- 5 Kısa tekrar özeti

1. Question 12 · Partial source

English: The supplied text contains only the end of Question 12: line 7 adds `sing + squawk` to `notes`, followed by printing `notes`. The answer choices are 11, 13, 23, 33, 50, or a compilation error on line 7.

Türkçe: Gönderilen metin Question 12'nin yalnız sonunu içeriyor: line 7, `notes` variable'ına `sing + squawk` ekliyor ve ardından `notes` yazdırılıyor. Seçenekler 11, 13, 23, 33, 50 veya line 7 nedeniyle compilation error'dır.

Appendix ile doğrulanan cevap: C — 23. Eksik başlangıçta `sing = 8, squawk = 2, notes = 0`dır. İlk iteration sonrasında değerler 7, 4 ve 11; ikinci iteration sonrasında 6, 6 ve 23 olur. `sing > squawk` artık false olduğu için loop biter. Attachment tam kodu içermediğinden burada kod gövdesi yeniden üretilmemiştir.

2. Questions 13–18

Question 13 · `do/while` syntax

```
boolean keepGoing = true;
int result = 15, meters = 10;
do {
    meters--;
    if (meters == 8) keepGoing = false;
    result -= 2;
} while keepGoing;
System.out.println(result);
```

English: What is the output? **Türkçe:** Çıktı nedir?

Cevap: G — Does not compile for a different reason. `do/while` condition'ı parentheses içinde olmalıdır: `while (keepGoing);`. Line 6 geçerlidir. Syntax düzeltilseydi body iki kez çalışır ve 11 yazdırırdı.

Question 14 · Enhanced `for` and `var`

```
for (var penguin : new int[2]) System.out.println(penguin);
var ostrich = new Character[3];
for (var emu : ostrich) System.out.println(emu);
List<Integer> parrots = new ArrayList<Integer>();
for (var macaw : parrots) System.out.println(macaw);
```

English: Which statements about the inferred element types are correct? **Türkçe:** Çıkarılan element type'ları hakkındaki hangi ifadeler doğrudur?

Cevap: B, D ve F. `int[]` elementi `int`, `Character[]` elementi `Character`, `List<Integer>` elementi `Integer`dir. Enhanced `for` içindeki `var`, iterable/array element type'ını compile time'da çıkarır.

Question 15 · Invalid case syntax

```
final char a = 'A', e = 'E';
char grade = 'B';
switch (grade) {
    default:
    case a:
    case 'B': 'C': System.out.print("great ");
    case 'D': System.out.print("good "); break;
    case e:
    case 'F': System.out.print("not good ");
}
```

English: What is the result? **Türkçe:** Sonuç nedir?

Cevap: F — None of the listed outputs. Does not compile. 'C': öncesinde `case` keyword'ü yoktur. Sorun case type uyumsuzluğu değildir. Satır `case 'B', 'C':` veya ayrı `case 'C':` olarak düzeltilseydi `great good` yazdırırdı.

Question 16 · Reverse traversal

English: Given `char[] wolf = {'W','e','b','b','y'}`, which loop snippets print the elements in reverse declaration order? **Türkçe:** Verilen `wolf` array'inin elementlerini declaration sırasının tersine hangi loop parçaları yazdırır?

Cevap: A, B ve D. Üçü de önce `index length - 1`, en son `index 0` okur. C ve F ilk olarak `wolf[wolf.length]` okuyup runtime'da `ArrayIndexOutOfBoundsException` fırlatır. E'de condition `r > -1` sabit true kalır ve `w` sürekli aynı değere atanır; infinite loop oluşur.

Question 17 · Loop final values

```
int participants = 4, animals = 2, performers = -1;
while ((participants = participants + 1) < 10) {}
do {} while (animals++ <= 1);
for (; performers < 2; performers += 2) {}
```

English: What distinct numbers are printed? **Türkçe:** Birbirinden farklı hangi sayılar yazdırılır?

Cevap: B ve E — 3 ve 10. `participants` condition false olduğunda 10'dur. `do/while` en az bir kez çalışır; postfix increment nedeniyle `animals` 3 olur. `performers`, `-1 → 1 → 3` ilerler. Son iki variable aynı distinct value olan 3'ü yazdırır.

Question 18 · Pattern matching and flow scoping

English: Which statements about pattern matching and flow scoping are correct? The choices mention `instance`, `instanceon`, `instanceof`, access after an `if`, compiler knowledge, and declaring a pattern variable in `else`. **Türkçe:** Pattern matching ve flow scoping hakkındaki hangi ifadeler doğrudur? Seçenekler doğru operator adını, `if` sonrası erişimi, compiler'ın type bilgisi ile `else` içinde pattern variable bildirmeyi sorgular.

Cevap: C ve E. Pattern matching `instanceof` ile uygulanır. Flow scope, pattern variable'ın yalnız compiler'ın `match`'in gerçekleştiğini kesin bildiği path'lerde accessible olmasıdır. Variable bazı

control-flow biçimlerinde `if` sonrasında kullanılabilir; `else` tek başına pattern declaration syntax'ı sunmaz.

3. Questions 19–24

Question 19 · Scope in `do/while`

```
double iguana = 0;
do {
    int snake = 1;
    System.out.print(snake++ + " ");
    iguana--;
} while (snake <= 5);
```

English: What is the output? **Türkçe:** Çıktı nedir?

Cevap: E — Does not compile. `snake` body block'unda bildirilmiştir ve `while` condition'ında scope dışındadır. Bu nedenle runtime çıktısı veya infinite loop analizi yapılmaz.

Question 20 · Labels, `break` and `continue`

English: Which pairs of statements can replace two empty statements in nested labeled loops so the code compiles and terminates? The labels are `L1` on a `while`, `L2` on a `do/while`, and `L3` on an infinite `for`. **Türkçe:** Nested labeled loop'lardaki iki empty statement yerine hangi statement çiftleri yazıldığında kod derlenir ve infinite loop'a girmez? `while`, `do/while` ve infinite `for` sırasıyla `L1`, `L2`, `L3` label'larına sahiptir.

Cevap: A ve E. A'daki `break L2`, innermost infinite loop'a ulaşmadan `do/while`dan çıkar. E'de line 8'deki `continue L2` bazı iteration'ları atlar; line 12'deki `continue L2` ise `L3` loop'undan `L2` condition'ına geçiş sağlar. C derlenmez çünkü `L3`, kendi loop'undan önceki line 8'de scope içinde değildir. B ve D sonunda infinite `for` içinde kalır.

Question 21 · Repairing a switch expression

```
void findZookeeper(Long id) {
    System.out.print(switch (id) {
        case 10 -> { "Jane" }
        case 20 -> { yield "Lisa"; };
        case 30 -> "Kelly";
        case 30 -> "Sarah";
        default -> "Unassigned";
    });
}
```

English: What is the minimum number of lines that must be corrected before the method compiles?

Türkçe: Method'un derlenmesi için en az kaç satır düzeltilmelidir?

Cevap: E — 4 lines. Line 22'de selector type `Long` switch için desteklenen type değildir. Line 23 block value üretmek için `yield "Jane";` ister. Line 24 block sonrasında fazladan semicolon içerir. Lines 25–26 duplicate case value kullanır; ikisinden biri değiştirilmelidir.

Question 22 · Constant case and fall-through

```
var tailFeathers = 3;
final var one = 1;
switch (tailFeathers) {
    case one: System.out.print(3 + " ");
    default: case 3: System.out.print(5 + " ");
}
while (tailFeathers > 1) {
    System.out.print(--tailFeathers + " ");
}
```

English: What is the output? **Türkçe:** Çıktı nedir?

Cevap: E — 5 2 1. `final var one = 1` compile-time constant'tır ve case label olabilir. Selector 3 olduğundan `case 3` çalışır ve 5 yazdırır. Loop iki kez çalışır; pre-decrement sırasıyla 2 ve 1 üretir.

Question 23 · Unmatched else

```
int penguin = 50, turtle = 75;
boolean older = penguin >= turtle;
if (older = true) System.out.println("Success");
else System.out.println("Failure");
else if (penguin != 50) System.out.println("Other");
```

English: What is the output? **Türkçe:** Çıktı nedir?

Cevap: F — None of the listed runtime results. Does not compile. Son `else if` ile eşleşebilecek bir `if` yoktur. Line 17'de comparison yerine assignment kullanılması syntactically geçerlidir ve expression `true` üretir; fakat program unmatched `else` nedeniyle derlenmez.

Question 24 · Invalid enhanced for syntax

```
for (var friend in friends) {
    System.out.println(friend);
}
```

English: Which possible types for `friends` allow the code to compile? **Türkçe:** `friends` için hangi olası type'lar kodun derlenmesini sağlar?

Cevap: G — None. Java enhanced `for` syntax'ı `in` değil colon (`:`) kullanır. `in` yerine `:` yazılıysaydı `Set`, `int[]` ve `Collection` uygun olabilirdi; `Map`, `String` ve `StringBuilder` doğrudan `Iterable` değildir.

4. Questions 25–29

Question 25 · String switch fall-through

```
String instrument = "violin";
final String CELLO = "cello";
String viola = "viola";
int p = -1;
switch (instrument) {
    case "bass": break;
    case CELLO: p++;
    default: p++;
    case "VIOLIN": p++;
    case "viola": ++p; break;
}
System.out.print(p);
```

English: What is the output? **Türkçe:** Çıktı nedir?

Cevap: D — 2. String case matching case-sensitive'dir; "violin", "VIOLIN" ile eşleşmez. `default` branch başlar ve `break` olmadığı için sonraki iki case body'ye fall through olur. `p`, -1'den üç increment ile 2'ye gelir. Non-final `viola` variable'ı case label olarak kullanılmadığından önemsizdir.

Question 26 · Infinite nested do/while

English: A `while` loop appends A; its nested `do/while` appends B and C while `r <= 1`. The variable `r` is incremented only after the inner loop. What happens? **Türkçe:** Bir `while` loop A; içindeki `do/while` ise `r <= 1` olduğu sürece B ve C ekler. `r` yalnız inner loop bittikten sonra artırılmaktadır. Ne olur?

Cevap: F — Derlenir fakat runtime'da sonlanmaz. `r` başlangıçta 1'dir ve inner loop içinde değişmez. Condition sürekli true kalır; execution hiçbir zaman outer body'nin `r++` satırına ulaşamaz.

Question 27 · Exhaustive switch-expression paths

```
byte amphibian = 1;
String name = "Frog";
String color = switch (amphibian) {
    case 1 -> { yield "Red"; }
    case 2 -> { if (name.equals("Frog")) yield "Green"; }
    case 3 -> { yield "Purple"; }
    default -> throw new RuntimeException();
};
```

English: What is printed? **Türkçe:** Ne yazdırılır?

Cevap: F — Does not compile. `case 2` block'unda `if` false olduğunda value üreten `yield` veya `throw` yoktur. Switch expression'daki her reachable case path bir value üretmeli veya tamamlanmadan exception fırlatmalıdır. Runtime'da selector 1 olacak olsa bile compiler tüm path'leri denetler.

Question 28 · Duplicate pattern variables and flow scope

```
void getFish(Object fish) {
    if (!(fish instanceof String guppy))
        System.out.print("Eat!");
    else if (!(fish instanceof String guppy)) {
        throw new RuntimeException();
    }
    System.out.print("Swim!");
}
```

English: What is the output of calling `getFish("goldie")`? **Türkçe:** `getFish("goldie")` çağrısının çıktısı nedir?

Cevap: F — None. Does not compile. İlk negated condition false olduğunda `guppy` sonraki `else` path'inde scope içindedir. İkinci `instanceof` aynı local name'i tekrar bildirdiğinden duplicate variable compilation error oluşur. İkinci name farklı olsaydı verilen input için `Swim!` yazdırılırdı.

Question 29 · do/while boundary

```
int y = -2;
do System.out.print(++y + " ");
while (y <= 5);
```

English: What is printed? **Türkçe:** Ne yazdırılır?

Cevap: C — -1 0 1 2 3 4 5 6. Pre-increment ilk olarak -1 üretir. Body 5'i yazdırdıktan sonra condition `5 <= 5` true olduğu için bir iteration daha gerçekleşir ve 6 yazdırılır. Ardından `6 <= 5` false olur.

5. Kısa tekrar özeti

- `do/while` body en az bir kez çalışır ve condition parentheses ister.
- Enhanced `for`: `for (Type item : arrayOrIterable);` Java'da `in` yoktur.
- Traditional switch statement fall-through yapabilir; arrow switch expression branch'leri value üretmelidir.
- Pattern variable yalnız compiler'ın match'i kesin bildiği flow path'inde scope içindedir.
- Label scope'u, label'ın bildirdiği statement ile sınırlıdır.
- Compile-time hata varsa runtime output veya exception seçeneğine geçme.

6. Teknik pekiştirme · Control-flow memory map**if önce type, sonra branch sorusudur**

Java condition için truthy/falsy conversion yapmaz; expression type kesinlikle `boolean` veya `Boolean`dan unbox edilebilir olmalıdır.

```
int count = 1;
// if (count) {}           // Does not compile
if (count > 0) {}          // Derlenir
```

Braces yoksa `else`, kendisinden önceki eşleşmemiş en yakın `if` bağlanır. Indentation compiler için anlam taşımaz.

Java 17 switch selector kartı

Standard Java 17 switch selector şu grupları destekler:

- `byte/Byte, short/Short, char/Character, int/Integer`
- `String`
- `enum`

`boolean, long, float` ve `double` switch selector olamaz. Wrapper selector `null` ise unboxing/matching sırasında `NullPointerException` oluşur. Standard Java 17 kapsamında `case null` kullanılamaz.

Case label sabitlik testi

Colon veya arrow kullanılmasından bağımsız olarak traditional case label:

- 1 Compile-time constant, enum constant veya String literal olmalı.
- 2 Selector type ile uyumlu olmalı.
- 3 Aynı switch içinde duplicate olmamalı.

`final int x = 2;` declaration sırasında constant expression ile initialize edilirse case label olabilir. `final int x; x = 2;` ise compile-time constant variable değildir.

Switch statement ve expression'ı karıştırma

Özellik	Statement	Expression
Sonuç value'su	Gerekmez	Her reachable path value/throw üretir
Colon form	Fall-through olabilir	Kullanılabilir; value için <code>yield</code> gerekir
Arrow form	Kullanılabilir	En yaygın form; fall-through yoktur
<code>break</code>	Switch'ten çıkar	Value üretmez
Exhaustiveness	Genelde gerekmez	Zorunlu

```
char grade = 'A';
int score = switch (grade) {
    case 'A' -> 100;
    case 'B' -> 80;
    default -> 0;
};
```

Memory tip: Switch **expression borçludur**: her yol bir value öder veya exception fırlatır.

Loop seçme ve scope kartı

Yapı	Condition zamanı	En az bir çalışma	Kaynak
<code>while</code>	Body'den önce	Hayır	Boolean condition
<code>do/while</code>	Body'den sonra	Evet	Boolean condition
Traditional <code>for</code>	Body'den önce	Hayır	init; condition; update
Enhanced <code>for</code>	Her element için	Hayır	Array veya <code>Iterable</code>

Traditional `for` header'ındaki init/update bölümleri birden fazla expression içerebilir. Bir declaration içinde farklı type'lar karıştırılmaz. Enhanced `for` index sağlamaz ve collection'ı structurally modify etmek için güvenli bir mekanizma değildir.

Label hedefi: `break` geniş, `continue` dar

- `break label;`, label'ın bağlı olduğu herhangi bir statement'tan çıkabilir.
- `continue label;` yalnız bir loop label'ına gidebilir ve o loop'un sonraki iteration'ına geçer.
- Unlabeled `break` en yakın loop veya switch'i; unlabeled `continue` en yakın loop'u hedefler.

Pattern matching için “true-path proof”

Pattern variable yalnız compiler o noktaya gelindiğinde match'in true olduğunu kanıtlayabiliyorsa scope içindedir.

```
if (obj instanceof String s && s.length() > 2) {
    System.out.println(s); // valid
}

if (!(obj instanceof String t)) return;
System.out.println(t);      // valid: buraya gelmek match'in true olduğunu kanıtlar
```

`obj instanceof String s || s.isEmpty()` derlenmez; sağ operand, match false iken de çalışabileceği için `s` kesin assigned değildir. `null instanceof T` her zaman false'tur ve exception fırlatmaz.

Reachability tuzağı

`for(;;)` açık bir infinite loop'tur. Compiler bazı açıkça unreachable statement'ları reddeder; örneğin `while (false) {}` body doğrudan unreachable olduğu için derlenmez. Buna karşılık `if (false) {}` Java tarafından özel olarak kabul edilir.

60 saniyelik active recall

- 1 `Long` neden switch selector olamaz ama `Integer` olabilir?
- 2 Switch expression'da `break value` yerine hangi keyword kullanılır?
- 3 `continue label` hangi tür statement'ı hedefleyebilir?
- 4 `instanceof` pattern variable `&&` sağında neden görünür, `||` sağında neden genellikle görünmez?
- 5 `do/while` ile `while` arasındaki tek garanti farkını söyle.

Hızlı kontrol

- 1 Java 17 selector listesi `Integer` içerir, `Long` içermez.

- 2 `yield`.
- 3 Yalnız loop statement'ını.
- 4 `&&` sağında left match true'dur; `||` sağında left false olabilir.
- 5 `do/while` body'yi en az bir kez çalıştırır.

Page coverage ledger

PDF page	Marker	Retained lines	Removed running-header lines	Status
0101	<!-- source-page: 0101 -->	12	0	complete
0102	<!-- source-page: 0102 -->	24	0	complete
0103	<!-- source-page: 0103 -->	33	1	complete
0104	<!-- source-page: 0104 -->	31	3	complete
0105	<!-- source-page: 0105 -->	33	1	complete
0106	<!-- source-page: 0106 -->	29	3	complete
0107	<!-- source-page: 0107 -->	33	1	complete
0108	<!-- source-page: 0108 -->	30	3	complete
0109	<!-- source-page: 0109 -->	35	1	complete
0110	<!-- source-page: 0110 -->	32	3	complete
0111	<!-- source-page: 0111 -->	32	1	complete
0112	<!-- source-page: 0112 -->	35	3	complete
0113	<!-- source-page: 0113 -->	36	1	complete
0114	<!-- source-page: 0114 -->	35	3	complete
0115	<!-- source-page: 0115 -->	38	1	complete

PDF page	Marker	Retained lines	Removed running-header lines	Status
0116	<!-- source-page: 0116 -->	46	3	complete
0117	<!-- source-page: 0117 -->	34	1	complete
0118	<!-- source-page: 0118 -->	34	3	complete
0119	<!-- source-page: 0119 -->	36	1	complete
0120	<!-- source-page: 0120 -->	38	3	complete
0121	<!-- source-page: 0121 -->	28	1	complete
0122	<!-- source-page: 0122 -->	38	3	complete
0123	<!-- source-page: 0123 -->	33	1	complete
0124	<!-- source-page: 0124 -->	35	3	complete
0125	<!-- source-page: 0125 -->	34	1	complete
0126	<!-- source-page: 0126 -->	30	3	complete
0127	<!-- source-page: 0127 -->	40	1	complete
0128	<!-- source-page: 0128 -->	34	3	complete
0129	<!-- source-page: 0129 -->	29	1	complete
0130	<!-- source-page: 0130 -->	40	3	complete

PDF page	Marker	Retained lines	Removed running-header lines	Status
0131	<!-- source-page: 0131 -->	36	1	complete
0132	<!-- source-page: 0132 -->	34	3	complete
0133	<!-- source-page: 0133 -->	30	1	complete
0134	<!-- source-page: 0134 -->	35	3	complete
0135	<!-- source-page: 0135 -->	30	1	complete
0136	<!-- source-page: 0136 -->	37	3	complete
0137	<!-- source-page: 0137 -->	36	1	complete
0138	<!-- source-page: 0138 -->	34	3	complete
0139	<!-- source-page: 0139 -->	29	1	complete
0140	<!-- source-page: 0140 -->	34	3	complete
0141	<!-- source-page: 0141 -->	11	1	complete
0142	<!-- source-page: 0142 -->	35	3	complete
0143	<!-- source-page: 0143 -->	39	1	complete
0144	<!-- source-page: 0144 -->	38	3	complete
0145	<!-- source-page: 0145 -->	36	1	complete

PDF page	Marker	Retained lines	Removed running-header lines	Status
0146	<!-- source-page: 0146 -->	37	3	complete
0147	<!-- source-page: 0147 -->	36	1	complete
0148	<!-- source-page: 0148 -->	39	3	complete
0149	<!-- source-page: 0149 -->	36	1	complete
0150	<!-- source-page: 0150 -->	40	3	complete
0151	<!-- source-page: 0151 -->	41	1	complete
0152	<!-- source-page: 0152 -->	37	3	complete
0153	<!-- source-page: 0153 -->	38	1	complete
0154	<!-- source-page: 0154 -->	42	3	complete

Coverage result: 0101–0154 aralığındaki 54/54 sayfanın marker'ı vardır; eksik veya yinelenen marker yoktur.